Objetivos –

De forma más concreta:

1. **Caracterización del modelo Robot Operating System 1 (ROS 1) integrado por Summit XL**

   Analizar la arquitectura de comunicación de ROS 1 integrada por la plataforma Summit XL, con el fin de identificar los procesos mas demansantes.

2. **Selección e integración de herramientas de medición**

   Investigar herramientas existentes para la medición de latencias y otros parámetros de rendimiento en ROS 1, seleccionar una herramienta adecuada e integrarla en el entorno experimental para la obtención de métricas cuantitativas.

3. **Diseño de arquitectura distribuida Robot–Edge**

   Diseñar una arquitectura distribuida mínima que represente un escenario Robot–Edge, constituida por nodos locales y nodos remotos desplegados en diferentes máquinas, permitiendo evaluar comunicaciones extremo a extremo en un entorno distribuido real.

4. **Evaluación bajo infraestructura Fifth Generation Mobile Network (5G)**

   Configurar y utilizar una infraestructura de red 5G representativa de un entorno real, considerando parámetros como latencia, variabilidad temporal y pérdida de paquetes, con el fin de generar escenarios experimentales reproducibles.

5. **Análisis cuantitativo de las comunicaciones**

   Medir y analizar cuantitativamente el comportamiento del sistema bajo distintas condiciones de red, evaluando parámetros como latencia end–to–end, tasa de transferencia, estabilidad de comunicación y diferencias entre ejecución local y distribuida.

Metodología – ...

Desarrollos teóricos realizados – ...

Desarrollo de prototipos y trabajo de laboratorio – ...

Resultados – ...

Líneas futuras – ...

Publicaciones – ...

Abstract –

Resumen –

Autor: Guillermo García Rodríguez, email: guillegr1999@gmail.com
Director: David Gomez-Barquero, email: dagobar@iteam.upv.es
Tutor externo: Nuria Molner Siurana, email: numolsiu@iteam.upv.es
Tutor externo: Raúl Lozano Teruel, email: raulote@iteam.upv.es
Fecha de entrega: 31-05-26

LISTA DE ACRÓNIMOS –

LISTA DE FIGURAS –

1. Interacción entre elementos de ROS [1] 9
2. Plataforma Summit XL 11
3. Arquitectura funcional de la red privada 5G Standalone (5G SA) integrada con infraestructura *Edge* y *Cloud*, donde se muestra la virtualización de funciones de red y aplicaciones, así como los mecanismos de orquestación [2]. 15
4. Frecuencia de publicación de los distintos subsistemas a lo largo del tiempo para un objetivo representativo. 16
5. Distribución de latencias por subsistema. 17
6. Distribución de densidad de la latencia end-to-end. 18
7. Evolución de la latencia end-to-end por objetivo. 18
8. Descomposición de la latencia end-to-end de cada objetivo en latencias de los subsistemas. 19

Índice

1. Introducción **6**
1.1. Contexto y motivación 6

2. Estado del arte **6**
2.1. Arquitecturas de comunicación para robótica conectada 6
2.2. Edge computing para sistemas robóticos 8
2.3. Middleware robótico: ROS 1 y ROS 2 8
2.4. Evaluación de latencia en sistemas robóticos distribuidos 10

3. Arquitectura del sistema propuesto **11**
3.1. Visión general de la arquitectura 11
3.2. Capa Robótica 11
3.2.1. Plataforma Hardware Summit XL 11
3.3. Capa de red 5G privada 13
3.3.1. Arquitectura general de la red privada 13

4. Análisis de latencia del sistema **16**
4.1. Análisis de latencia del sistema centralizado 16
4.1.1. Procesamiento y modelado de los datos 16
4.1.2. Resultados del análisis de latencia 17
4.1.3. Conclusiones e implicaciones 20

A. Análisis completo del sistema **21**

B. Análisis de la latencia en arquitectura centralizada **42**

1. Introducción

1.1. Contexto y motivación

La robótica móvil está experimentando un crecimiento significativo en sectores como la industria o la logística. Los robots deben operar en entornos dinámicos y procesar grandes cantidades de información procedente de múltiples sensores. Tradicionalmente, los sistemas robóticos han sido diseñados para realizar las tareas computacionales de forma completamente local. Sin embargo, la creciente complejidad de las tareas ha puesto de manifiesto las limitaciones de esta estrategia, lo que ha impulsado el desarrollo de arquitecturas de computación distribuida que asisten con el procesamiento más allá de las limitaciones de los sistemas embarcados.

El paradigma de *edge computing* ha emergido como una alternativa que permite trasladar parte del procesamiento a servidores cercanos, reduciendo la carga computacional del robot.

Paralelamente, las redes 5G ofrecen características especialmente adecuadas para sistemas robóticos distribuidos. Estas capacidades permiten establecer arquitecturas en las que los robots pueden intercambiar grandes volúmenes de datos con infraestructuras de computación externas en tiempo casi real, facilitando la distribución de tareas entre el robot y los servidores de *edge computing*.

En este contexto, surge la necesidad de investigar metodologías que permitan distribuir de forma eficiente las funciones de un sistema robótico entre el robot y el *edge*. Este tipo de arquitecturas híbridas plantea diversos desafíos, entre ellos la gestión de la latencia de comunicación, la selección de las tareas adecuadas para su ejecución remota y la integración de estas capacidades dentro de los middleware robóticos existentes.

El presente trabajo explora la integración de mecanismos de distribución de funciones entre el robot y el *edge* en un sistema robótico basado en ROS desplegado sobre una red privada 5G.

2. Estado del arte

El desarrollo de sistemas robóticos conectados requiere la integración de infraestructuras de comunicaciones avanzadas, arquitecturas de computación distribuida y middleware robótico. En este capítulo se revisan los principales trabajos relacionados con redes 5G (Sección 2.1), *edge computing* (Sección 2.2), *middleware* robótico (Sección 2.3) y evaluación de latencia en sistemas robóticos distribuidos (Sección 2.4).

2.1. Arquitecturas de comunicación para robótica conectada

El desarrollo del campo de la robótica móvil esta estrechamente vinculado al desarrollo de infraestructuras de comunicación capaces de soportar sistemas distribuidos y aplicaciones en tiempo real. Tradicionalmente, los robots móviles realizaban la mayor parte del

procesamiento de forma local. Sin embargo, el creciente uso de sensórica, algoritmos de percepción basados en inteligencia artificial (IA) y sistemas de control remoto ha incrementado significativamente las necesidades de computo.

En este contexto, las redes móviles 5G han emergido como una tecnología habilitadora de nuevas arquitecturas de robótica conectada. A diferencia de generaciones anteriores de redes móviles, el marco conceptual de 5G definido en IMT-2020 establece diferentes escenarios de servicio orientados a soportar aplicaciones con altos requisitos de capacidad, baja latencia y conectividad masiva de dispositivos [3]. Estas capacidades permiten que robots móviles transmitan grandes volúmenes de datos procedentes de sensores y reciban comandos de control con tiempos de respuesta reducidos.

Desde el punto de vista arquitectónico, el sistema 5G se basa en un modelo denominado Service–Base Architecture (SBA), en el que las funciones de red se implementan como servicios modulares. Este enfoque facilita la virtualización de la red y la distribución flexible de funciones. Además, el sistema 5G introduce mecanismos como *network slicing*, que permiten crear redes lógicas con características específicas para distintos servicios, y un modelo avanzado de Quality of Service (QoS) basado en identificadores 5G QoS Identifier (5QI) para garantizar requisitos de latencia, fiabilidad y prioridad del tráfico. Asimismo, el estándar contempla el despliegue de *Non-Public Networks* (NPN), que permiten operar infraestructuras 5G dedicadas para aplicaciones críticas [3].

Un ejemplo de red privadas 5G es la desplegada en el campus de la Universitat Politècnica de València, que proporciona cobertura interior y exterior mediante múltiples bandas de frecuencia y diferentes implementaciones del núcleo 5G, permitiendo la evaluación de diversos escenarios experimentales de conectividad [2]. La red 5G SA del Instituto de Telecomunicaciones y Aplicaciones Multimedia (iTEAM-UPV) se describe con mayor detalle en la sección 3.3

El uso de redes 5G también facilita la transición hacia arquitecturas robóticas distribuidas, en las que determinadas funciones del sistema se ejecutan fuera de la plataforma robótica. Este enfoque permite reducir los requisitos de *hardware* embarcado en el robot y mejorar la escalabilidad del sistema, aunque introduce nuevos retos relacionados con la latencia de comunicación, la estabilidad de los enlaces y la fiabilidad global de la red.

Diversos trabajos experimentales han explorado estas arquitecturas mediante plataformas de prueba que integran robots móviles con redes 5G privadas. En estos entornos, los robots transmiten flujos de datos sensoriales al *edge* para su procesamiento y reciben información procesada para la toma de decisiones o la teleoperación. Los resultados muestran que el rendimiento del sistema depende de múltiples fuentes de latencia, incluyendo la transmisión de vídeo, el procesamiento de percepción asistido por IA y el envío de comandos de control al robot [4].

En conjunto, estas investigaciones muestran que las redes 5G proporcionan la infraestructura necesaria para soportar arquitecturas robóticas distribuidas. No obstante, el comportamiento global del sistema depende de la interacción entre la red de comunicaciones, la arquitectura de procesamiento distribuido y el software robótico empleado. Por este motivo, el análisis del rendimiento de estos sistemas requiere considerar de forma

conjunta las características de la red de comunicaciones y la distribución funcional de los componentes del sistema robótico.

2.2. Edge computing para sistemas robóticos

El incremento en la cantidad de datos generados por sistemas robóticos modernos ha impulsado el desarrollo de arquitecturas de procesamiento distribuidas que permitan complementar las capacidades computacionales de los robots móviles. La ejecución local de estas tareas puede resultar limitada por las limitaciones de potencia de procesamiento y consumo energético de los sistemas embarcados.

En este contexto, el paradigma de *edge computing* propone trasladar parte del procesamiento de datos desde los dispositivos finales hacia nodos de computación situados en el borde de la red. Estos nodos, denominados *edge servers*, permiten ejecutar tareas computacionalmente más demandantes. Su proximidad a la fuente de datos reduce la latencia de comunicación respecto a los centros de datos (*cloud*), lo que se traduce en menores tiempos de respuesta para aplicaciones sensibles al retardo.

Este tipo de configuraciones suele describirse como arquitecturas *Robot–Edge–Cloud*, en las que las tareas de control y adquisición de datos se ejecutan en el robot, mientras que el procesamiento intensivo se desplaza hacia nodos de computación en el borde de la red o infraestructuras *cloud*. Este enfoque permite reducir la carga computacional en la plataforma robótica, facilita el despliegue de aplicaciones más complejas y mejora la escalabilidad del sistema.

La integración de capacidades de computación en el borde de la red ha sido considerada en la evolución de las arquitecturas 5G . En particular, los trabajos de estandarización del 3rd Generation Partnership Project (3GPP) han analizado diferentes arquitecturas para habilitar aplicaciones desplegadas en el borde de la red y facilitar la interacción entre los dispositivos finales y los servidores de *edge computing* [5].

Plataformas experimentales basadas en redes privadas 5G han demostrado la viabilidad de integrar robots móviles con infraestructuras de computación distribuida en entornos industriales. Esto abre nuevas posibilidades para la optimización de procesos [6].

No obstante, la adopción de arquitecturas robóticas basadas en *edge computing* plantea diversos retos técnicos, entre los que destacan la gestión de la latencia de comunicación, la asignación dinámica de recursos de computación y la coordinación entre los distintos componentes distribuidos del sistema. Por este motivo, el diseño de sistemas robóticos distribuidos requiere considerar conjuntamente las capacidades de la red de comunicaciones, la arquitectura de procesamiento y los requisitos temporales de las aplicaciones robóticas.

2.3. Middleware robótico: ROS 1 y ROS 2

El desarrollo de sistemas robóticos modernos requiere de software capaz de gestionar la interoperabilidad entre componentes heterogéneos. En este contexto, los *middleware* robóticos proporcionan una capa de abstracción que facilita la integración de elementos

dentro de una arquitectura modular. Entre las distintas soluciones propuestas, Robot Operating System (ROS) se ha consolidado como uno de los marcos de desarrollo más utilizados en investigación y aplicaciones robóticas.

ROS proporciona un conjunto de herramientas y bibliotecas para la construcción de aplicaciones robóticas mediante la orquestación de módulos independientes denominados *nodes*. Estos nodos intercambian información a través de un modelo de comunicación basado en el paradigma *publish–subscribe*, en el que los productores de datos publican información en canales denominados *topics*, mientras que los consumidores se suscriben a dichos canales para recibir los mensajes correspondientes. Este modelo facilita el desacoplamiento entre los distintos componentes del sistema y permite desarrollar arquitecturas robóticas distribuidas [1]. La figura 1 muestra la relación de estos elementos.

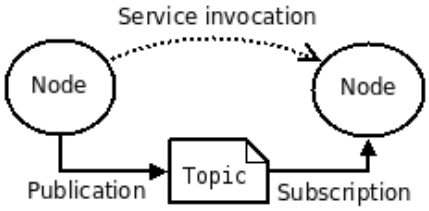


Figura 1: Interacción entre elementos de ROS [1]

La primera generación del sistema, conocida como ROS 1, la comunicación entre nodos se implementa mediante mecanismos de transporte basados en los protocolos de red Transmission Control Protocol (TCP) y User Datagram Protocol (UDP), a través de las implementaciones TCPROS y UDPROS. Aunque esta arquitectura ha demostrado ser eficaz, presenta ciertas limitaciones en términos de escalabilidad y soporte para aplicaciones con requisitos estrictos de tiempo real. En particular, la latencia de comunicación entre nodos puede verse afectada por factores como el tamaño de los mensajes, la carga del sistema o las características de la red subyacente [7].

Con el objetivo de superar estas limitaciones, la segunda generación del *framework*, Robot Operating System 2 (ROS 2), introduce una arquitectura de comunicación basada en el estándar Data Distribution Service (DDS). Este middleware, orientado a sistemas distribuidos, implementa de forma nativa el modelo *publish–subscribe* y proporciona mecanismos avanzados de gestión de políticas de QoS, como la configuración de políticas de latencia [1]. La adopción de DDS permite así mejorar la escalabilidad de los sistemas robóticos y facilita su integración en entornos distribuidos complejos, al tiempo que posibilita adaptar el comportamiento de la comunicación a los requisitos específicos de cada aplicación.

En consecuencia, el uso de ROS 2 es adecuado para el desarrollo de sistemas robóticos distribuidos. No obstante, el comportamiento temporal de estos sistemas depende de múltiples factores, incluyendo la configuración del middleware, la carga computacional de los nodos y las características de la red de comunicaciones. Por ello, el análisis del rendimiento y de la latencia en sistemas robóticos basados en ROS constituye un aspecto

fundamental en el diseño y evaluación de arquitecturas robóticas distribuidas. Los avances en este área se tratan en la siguiente sección 2.4.

2.4. Evaluación de latencia en sistemas robóticos distribuidos

En el modelo *publish–subscribe* los mensajes son transmitidos a través de distintos niveles del *middleware* y de la pila de red. Este proceso introduce diferentes fuentes de latencia que pueden afectar al comportamiento global de la aplicación robótica. Entre estas fuentes se incluyen el tiempo de serialización de los mensajes, la gestión de colas de publicación y suscripción, la planificación de procesos en el sistema operativo y el retardo introducido por la red.

Diversos trabajos han analizado el impacto de estos factores en el rendimiento de aplicaciones robóticas. En particular, Nishimura et al. proponen la herramienta ROS-Aware Publish/Subscribe Latency Evaluation Tool (RAPLET) para analizar la latencia de comunicación entre nodos en aplicaciones basadas en ROS [7]. Esta herramienta permite descomponer la latencia total en distintos componentes asociados al procesamiento de los nodos, la planificación de hilos en el sistema operativo y la transmisión de datos a través de la pila de red. Los resultados experimentales muestran que la latencia de comunicación puede variar significativamente en función de factores como el tamaño de los mensajes, la carga computacional del sistema o la configuración del *hardware* utilizado. En aplicaciones robóticas complejas, en las que múltiples nodos intercambian grandes volúmenes de información sensorial, estos retardos pueden acumularse y afectar al comportamiento temporal del sistema.

Estudios recientes han analizado el comportamiento de este tipo de arquitecturas en entornos experimentales que combinan robots móviles, redes de comunicaciones avanzadas y plataformas de computación distribuida. Por ejemplo, en plataformas robóticas conectadas a redes móviles de nueva generación, la latencia total del sistema puede estar influenciada por múltiples componentes, incluyendo la transmisión de vídeo, el procesamiento de algoritmos de percepción y el envío de comandos de control al robot [4].

En consecuencia, la evaluación de la latencia en sistemas robóticos distribuidos constituye un elemento clave para el diseño y la validación de arquitecturas robóticas. El análisis de los distintos componentes que contribuyen al retardo total permite identificar cuellos de botella en el sistema y optimizar la distribución de funciones entre los distintos elementos de la arquitectura.

3. Arquitectura del sistema propuesto

3.1. Visión general de la arquitectura

3.2. Capa Robótica

3.2.1. Plataforma Hardware Summit XL

La capa robótica está constituida por la plataforma móvil Summit XL, desarrollada por Robotnik Automation, que actúa como elemento físico principal del sistema experimental. Se trata de una base móvil de propósito general orientada a aplicaciones de investigación y vigilancia en entornos interiores y exteriores. Esta capa integra los subsistemas mecánicos, de control, computación embarcada y sensórica necesarios para la operación tanto teleoperada como autónoma. En la Fig. 2 se muestran diferentes vistas de la plataforma.

(a) Vista frontal **(b)** Vista lateral **(c)** Vista trasera

Figura 2: Plataforma Summit XL

El Summit XL dispone de cuatro ruedas motrices con tracción diferencial tipo *skid-steering*, donde el giro se produce mediante deslizamiento controlado provocado por la diferencia de velocidad entre los laterales derecho e izquierdo. Tiene una elevada capacidad de tracción y robustez mecánica.

El sistema de tracción está compuesto por:

- Cuatro servomotores *brushless* de 500 W.
- Encoders integrados para realimentación de velocidad y posición.
- Controladores de motor interconectados mediante bus Controller Area Network (CAN).

La plataforma alcanza una velocidad máxima de 3 m/s y puede superar pendientes de hasta el 80 %, lo que la hace adecuada para escenarios exigentes.

Las principales especificaciones físicas del robot se recogen en la Tabla 1.

Dimensiones	720 × 614 × 416 mm
Peso	65 kg
Capacidad de carga	50 kg
Protección	IP50
Rango de temperatura	-10 °C a +45 °C

Tabla 1: Especificaciones técnicas principales

El sistema de alimentación se basa en una batería $LiFePO_4$ de 15 Ah a 48 V, proporcionando una autonomía de hasta 10 horas en función del perfil de operación.

La plataforma integra un PC industrial embarcado basado en Linux que actúa como unidad central de procesamiento, coordinando el control de actuadores y la adquisición de datos sensoriales. La arquitectura *software* se basa en ROS 1. La arquitectura basada en nodos permite desacoplar los módulos de percepción, localización, planificación y control, habilitando su ejecución local o su externalización hacia la infraestructura Edge a través de la red 5G privada.

La comunicación entre el ordenador embarcado y los drivers de motor se realiza a través del bus CAN, lo que garantiza comunicación determinista y baja latencia en el control de actuadores.

En términos de conectividad, la plataforma dispone de:

- WiFi 802.11n (WiFi-4).
- Conectividad interna: USB, RS232 y GPIO.
- Conectividad externa: USB, RJ45 y 12 VDC.

Estas interfaces permiten la integración directa con la infraestructura de red 5G privada, descrita en la sección 3.3, facilitando la transmisión de datos de percepción y control hacia nodos *Edge* distribuidos.

La plataforma admite la integración de distintos sensores en función del caso de uso. En la configuración empleada en este trabajo se consideran:

- Unidad de medición inercial (IMU).
- Cámara Pan-Tilt-Zoom (PTZ) para transmisión de vídeo en tiempo real.
- Sensor Light Detection and Ranging (LiDAR) para percepción del entorno.

De este modo, la plataforma robótica constituye el origen de los flujos de datos de percepción y control que serán procesados localmente o externalizados según la estrategia de *offloading* definida en la arquitectura global del sistema, integrando la capa física con la infraestructura de computación distribuida descrita en capítulos anteriores.

3.3. Capa de red 5G privada

La capa de red privada constituye la columna vertebral del entorno experimental, proporcionando conectividad inalámbrica de baja latencia, alto ancho de banda y capacidades avanzadas de segmentación. Esta infraestructura se basa en la red privada 5G desplegada en la Universidad Politécnica de Valencia (UPV), concebida como entorno de experimentación para aplicaciones avanzadas de *Edge Computing*, *Digital Twins* e Internet of Things (IoT) [8] [2].

El diseño de esta capa no se limita a ofrecer conectividad radio, sino que integra capacidades de computación en *edge*, virtualización de funciones de red y mecanismos de orquestación inteligente, alineándose con la evolución hacia arquitecturas 5G-Advanced (5GA) [8]

3.3.1. Arquitectura general de la red privada

La red privada 5G del iTEAM-UPV constituye la infraestructura de comunicaciones sobre la que se soporta la capa robótica descrita en la Sección 3.2. Esta red presenta una arquitectura completa en modo 5G SA, con despliegue *indoor* y *outdoor*, múltiples bandas de frecuencia y diversas implementaciones de núcleo 5G [2].

Desde el punto de vista funcional, la arquitectura se estructura en cuatro bloques principales:

1. **Radio Access Network (RAN)**

   La Radio Access Network (RAN) es el subsistema encargado de proporcionar conectividad radio entre los User Equipments (UEs) y el núcleo 5G. Entre las características más relevantes del equipamiento empleado destacan:

   - Ancho de banda de hasta 100 MHz.
   - Configuración flexible en *Time Division Duplex* (TDD).
   - Soporte MIMO 4x4.
   - Arquitectura funcionalmente separada Centralized Unit – Distributed Unit – Remote Unit (CU–DU–RU).

   La separación funcional CU–DU–RU permite desacoplar el procesamiento de la capa física de las capas superiores, reducir la latencia situando la Descentralized Unit (DU) próxima a la unidad radio y virtualizar la Centralized Unit (CU) sobre infraestructura de propósito general. Asimismo, el soporte de *radio slicing* mediante asignación dinámica de recursos físicos (PRBs) permite priorizar determinados flujos de tráfico en función de los requisitos de cada aplicación.

   En aplicaciones de robótica móvil, la RAN resulta crítica al garantizar un *uplink* robusto para la transmisión de datos sensoriales y minimizar el *jitter* en escenarios de teleoperación.

2. **Núcleo 5G (5G Core)**

   El núcleo 5G implementa las funciones del plano de control (*Control Plane*) y del plano de usuario (*User Plane*). El núcleo sigue una arquitectura SBA, propia de 5G SA, lo que permite la virtualización y desacoplamiento de funciones de red sobre infraestructura de propósito general.

   Estas funciones pueden ejecutarse de forma virtualizada tanto en infraestructura *Edge* como en *Cloud*, tal y como se representa en la Figura 3.

   La posibilidad de instanciar múltiples User Platform Function (UPF) habilita mecanismos de *network slicing* a nivel de *core*, permitiendo:

   - Aislar tráfico por aplicación.
   - Implementar políticas de QoS diferenciadas.
   - Aproximar el plano de usuario al *Edge*.

   En sistemas distribuidos basados en ROS con estrategias de *offloading*, la proximidad de la UPF a los nodos *Edge* reduce la latencia End to End (E2E) en la transmisión de *topics* de alta tasa, lo que resulta determinante en aplicaciones sensibles al retardo.

3. **Infraestructura Edge asociada al plano de usuario**

   La infraestructura *Edge* constituye el punto intermedio del *continuum Robot–Edge–Cloud* [8]. Asociada al plano de usuario, permite ejecutar tareas de procesamiento con requisitos estrictos de latencia.

   La infraestructura *edge* puede alojar instancias virtualizadas de la UPF, aproximando el plano de usuario a los dispositivos finales y reduciendo la latencia E2E. Esta integración refuerza el carácter distribuido y programable de la arquitectura, alineado con el paradigma de Network Function Virtualization (NFV).

   En el contexto de robótica móvil, el *edge* posibilita:

   - Teleoperación inmersiva.
   - Procesamiento en tiempo real de vídeo.
   - *Offloading* de algoritmos de Simultaneous Localization and Mapping (SLAM) de elevada complejidad computacional.
   - Inferencia de modelos de inteligencia artificial.

   Este enfoque reduce la carga computacional del robot y evita el envío continuo de grandes volúmenes de datos al *cloud*.

4. **Interconexión con cloud**

   La interconexión con infraestructuras *cloud* permite ejecutar tareas computacionalmente intensivas no críticas en latencia, tales como:

- Procesamiento Big Data.
- Entrenamiento de modelos de aprendizaje automático.
- Almacenamiento histórico de datos.

Este diseño desacopla los procesos sensibles a la latencia (gestionados en *edge*) de aquellos con mayores requerimientos computacionales pero menor criticidad temporal (gestionados por el *cloud*) [2].

Como se muestra en la Figura 3, la arquitectura integra la red privada 5G con la infraestructura *edge* y la interconexión *cloud* bajo un esquema de orquestación dual (*Network Orchestration* y *Edge Orchestration*). Esta separación permite la gestión dinámica de recursos, la instanciación de funciones de red virtualizadas y el despliegue automatizado de aplicaciones distribuidas.

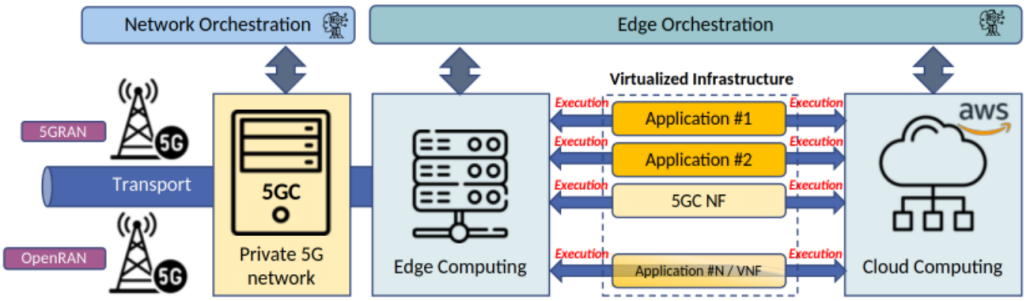


Figura 3: Arquitectura funcional de la red privada 5G SA integrada con infraestructura *Edge* y *Cloud*, donde se muestra la virtualización de funciones de red y aplicaciones, así como los mecanismos de orquestación [2].

La adopción de una red privada 5G como capa de red del sistema experimental responde a la necesidad de garantizar aislamiento lógico, control determinista de QoS y segmentación mediante *network slicing*, capacidades que no están disponibles de forma nativa en tecnologías inalámbricas convencionales como IEEE 802.11 (WiFi), especialmente en entornos con tráfico heterogéneo y requisitos de baja latencia.

Desde una perspectiva sistémica, la red privada no actúa únicamente como medio de transporte de datos, sino como un componente activo del sistema distribuido, alineado con el paradigma *Robot–Edge–Cloud* y con los requisitos de aplicaciones avanzadas como Digital Twins y teleoperación inmersiva.

4. Análisis de latencia del sistema

4.1. Análisis de latencia del sistema centralizado

4.1.1. Procesamiento y modelado de los datos

El análisis de latencias se ha realizado mediante la construcción de un modelo secuencial del sistema, representado como un pipeline de eventos: *Decisión* → *Arbitraje* → *Ejecución* → *Percepción*. Este modelo permite descomponer la latencia end-to-end en contribuciones parciales por subsistema.

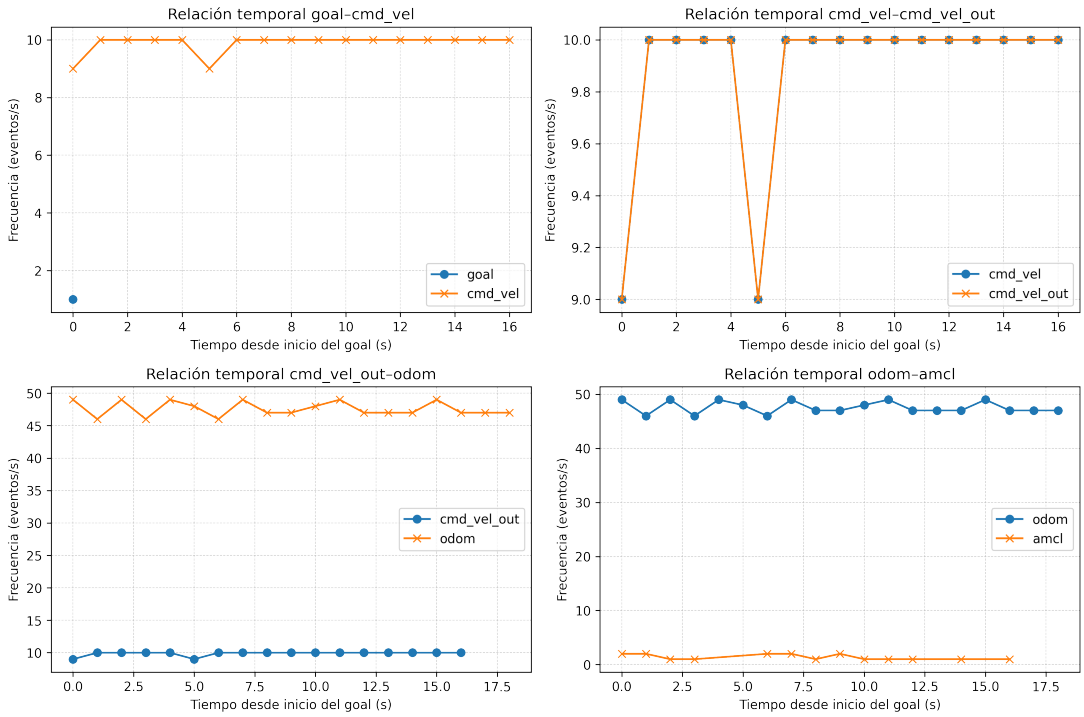


Figura 4: Frecuencia de publicación de los distintos subsistemas a lo largo del tiempo para un objetivo representativo.

No obstante, esta representación constituye una simplificación del comportamiento real del sistema. En la práctica, los distintos subsistemas operan de forma parcialmente desacoplada, con dependencias que se aproximan mejor a una estructura de grafo acíclico dirigido (DAG), donde múltiples flujos de información evolucionan de manera asíncrona. La interpretación como un modelo lineal event-driven se adopta, por tanto, como una aproximación práctica que permite estructurar el análisis, asumiendo una relación causal dominante entre eventos.

Esta asincronía se pone de manifiesto en la Figura 4, donde se observa un desacoplamiento en las frecuencias de publicación de los distintos subsistemas, especialmente en los tramos *cmd_vel_out–odom* y *odom–amcl*, donde existen relaciones de tipo uno-a-muchos

y muchos-a-uno. Este comportamiento evidencia la naturaleza asíncrona del sistema. Como consecuencia, la selección de eventos basada en el criterio de primer evento posterior puede no capturar adecuadamente la correspondencia temporal entre señales.

Para mitigar este efecto, se aplica un tratamiento selectivo de las series temporales mediante *Dynamic Time Warping* (DTW), un algoritmo que mide la similitud entre secuencias permitiendo deformaciones no lineales en el eje temporal. Este enfoque resulta especialmente adecuado cuando existen discrepancias en la frecuencia de eventos, ya que proporciona una alineación temporal más consistente entre subsistemas.

4.1.2. Resultados del análisis de latencia

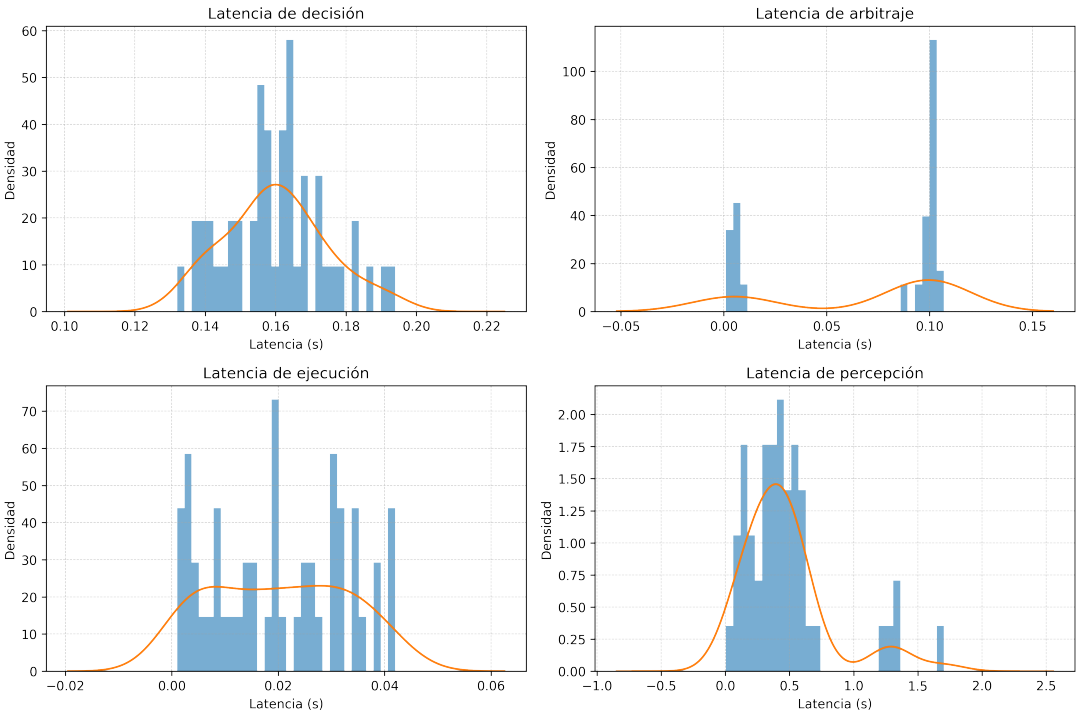


Figura 5: Distribución de latencias por subsistema.

La Figura 5 muestra la distribución de latencias por subsistema, evidenciando comportamientos diferenciados que reflejan la naturaleza funcional de cada módulo. El subsistema de decisión presenta una distribución aproximadamente unimodal y relativamente concentrada en torno a 0.16 s, con una dispersión moderada. Este patrón es consistente con un proceso determinista con carga computacional estable, donde la variabilidad observada puede atribuirse principalmente a pequeñas fluctuaciones en la disponibilidad de datos de entrada o planificación interna. Por el contrario, el subsistema de arbitraje exhibe una distribución claramente multimodal, con agrupaciones bien definidas en distintos rangos de latencia. Este comportamiento no es ruido, sino indicativo de la existencia de múltiples caminos de ejecución o políticas de selección de comandos, lo que sugiere que el

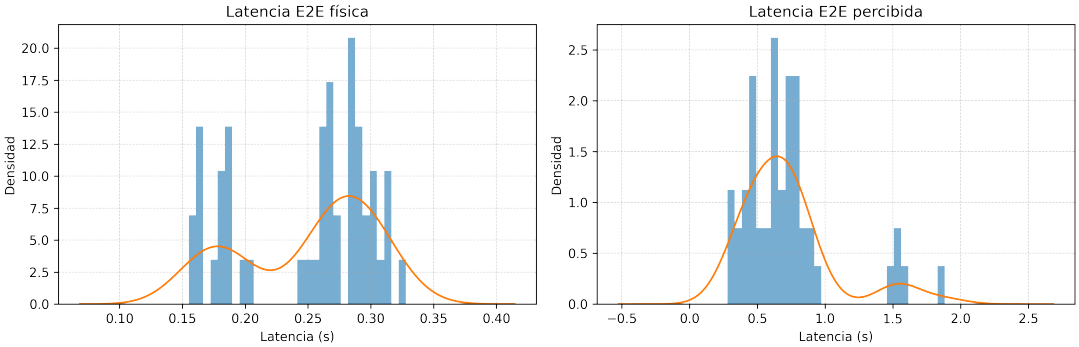


Figura 6: Distribución de densidad de la latencia end-to-end.

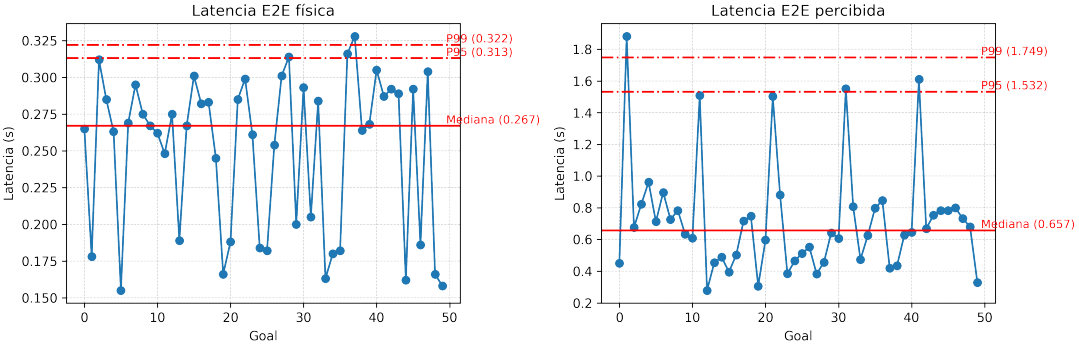


Figura 7: Evolución de la latencia end-to-end por objetivo.

coste temporal depende fuertemente del contexto operativo (por ejemplo, complejidad del conflicto a resolver). El subsistema de ejecución, aunque presenta valores medios muy reducidos (media = 0.0200 s), muestra una distribución irregular y dispersa, sin una forma claramente gaussiana. Esto apunta a un proceso ligero pero sensible a factores externos, como la sincronización con el sistema de control, introduciendo jitter aunque acotado en magnitud. Finalmente, el subsistema de percepción concentra la mayor complejidad temporal del sistema. Su distribución presenta un núcleo dominante en el rango 0.4–0.6 s, acompañado de eventos aislados de alta latencia que generan una cola pesada. Este patrón es característico de procesos intensivos en computación y dependientes de entrada sensorial, donde la carga varía dinámicamente (por ejemplo, en función de la complejidad del entorno). Los percentiles superiores (P95 = 1.295 s, P99 = 1.528 s) confirman la presencia de episodios de degradación puntual, lo que convierte a este subsistema en el principal contribuyente a la variabilidad global del sistema. Estos resultados se sintetizan en la Tabla 2, donde se observa que la variabilidad de la latencia de percepción es del mismo orden que la de la latencia end-to-end percibida, tanto en desviación estándar como en percentiles altos. Aunque la latencia end-to-end incorpora contribuciones adicionales de otros subsistemas, la similitud en términos de dispersión indica que el módulo de percepción constituye el principal factor que determina la variabilidad global del sistema.

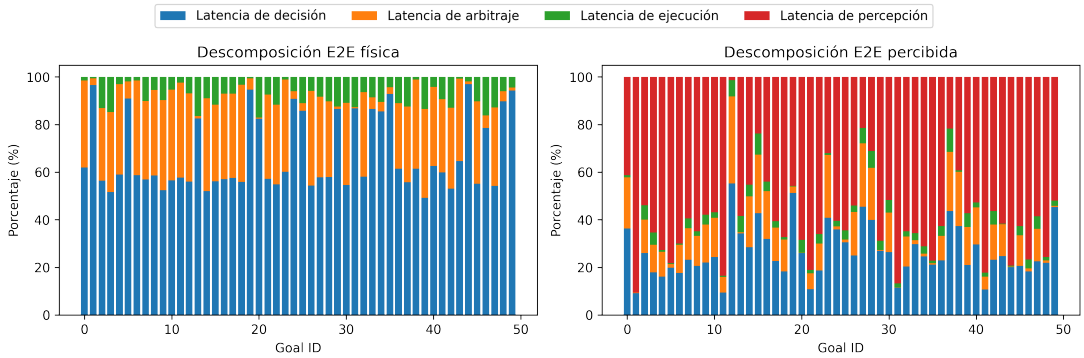


Figura 8: Descomposición de la latencia end-to-end de cada objetivo en latencias de los subsistemas.

Métrica	$L_{decision}$	$L_{arbitraje}$	$L_{ejecucion}$	$L_{percepcion}$	$L_{E2Efisica}$	$L_{E2Epercibida}$
Media	0.1602	0.0693	0.0200	0.4684	0.2495	0.7179
Desv. std	0.0143	0.0446	0.0127	0.3502	0.0531	0.3457
Mínimo	0.1320	0.0010	0.0010	0.0040	0.1550	0.2790
P50	0.1605	0.0990	0.0200	0.4180	0.2670	0.6570
P90	0.1784	0.1011	0.0362	0.7499	0.3041	1.0151
P95	0.1852	0.1031	0.0401	1.2951	0.3131	1.5316
P99	0.1925	0.1055	0.0415	1.5281	0.3221	1.7487
Máximo	0.1940	0.1070	0.0420	1.7040	0.3280	1.8820

Tabla 2: Estadísticos descriptivos y percentiles de latencia por subsistema y latencias E2E (en segundos).

Por el contrario, la latencia percibida presenta una mayor dispersión y una estructura heterogénea, con un núcleo principal en torno a 0.5–0.8 s y la aparición de eventos de alta latencia. La mediana se sitúa en 0.657 s, mientras que los valores extremos alcanzan 1.882 s. Los percentiles altos (P95 = 1.531 s, P99 = 1.748 s) evidencian la existencia de episodios de degradación temporal claramente diferenciados del comportamiento nominal.

Estos resultados se sintetizan en la Tabla 2, donde se observa que la variabilidad de la latencia de percepción es comparable a la de la latencia end-to-end percibida, lo que indica que este subsistema actúa como principal fuente de variabilidad del sistema.

Finalmente, la Figura 7 muestra la evolución de la latencia end-to-end por objetivo. La latencia física presenta variaciones acotadas dentro de un rango estrecho, aunque con cambios irregulares entre objetivos, lo que indica un comportamiento estable en magnitud pero no completamente uniforme. Por el contrario, la latencia percibida muestra una mayor variabilidad, con oscilaciones pronunciadas y la aparición de picos aislados de alta latencia, coherentes con los valores extremos observados en las distribuciones.

La Figura 8 muestra la descomposición porcentual de la latencia end-to-end para cada objetivo. En el caso de la latencia física, la contribución está dominada por los módulos de decisión y arbitraje, con una participación consistente y estable entre objetivos, mientras

que la ejecución presenta un impacto marginal. Este comportamiento refleja un pipeline de control determinista y con baja variabilidad. Por el contrario, en la latencia percibida se observa un cambio estructural claro: el módulo de percepción pasa a ser el principal contribuyente, alcanzando en numerosos casos más del 70 % de la latencia total. Además, su contribución presenta una elevada variabilidad entre objetivos. Estos resultados evidencian que el incremento de latencia y variabilidad observado en el sistema no se origina en el pipeline de control, sino en el proceso de percepción, que actúa como principal cuello de botella del sistema.

En conjunto, los resultados indican que la diferencia entre latencia física y percibida no es únicamente de magnitud, sino también de naturaleza estadística. Mientras que la primera presenta un comportamiento estable, la segunda está caracterizada por una mayor dispersión, asimetría y presencia de eventos extremos, lo que evidencia una fuente de variabilidad estructural asociada al proceso de percepción.

4.1.3. Conclusiones e implicaciones

El análisis realizado permite extraer una serie de conclusiones claras sobre el comportamiento temporal del sistema.

En primer lugar, el pipeline de control (decisión, arbitraje y ejecución) presenta un comportamiento estable y acotado, tanto en términos de magnitud como de variabilidad. La latencia física se mantiene en un rango estrecho, con baja dispersión y sin presencia de eventos extremos, lo que indica un funcionamiento determinista y predecible de estos subsistemas.

En contraste, el módulo de percepción introduce un cambio cualitativo en el comportamiento del sistema. No solo incrementa significativamente la latencia end-to-end, sino que además actúa como principal fuente de variabilidad. La similitud entre la dispersión de la latencia de percepción y la latencia end-to-end percibida, tanto en desviación estándar como en percentiles altos, evidencia que este subsistema domina el comportamiento global del sistema.

Adicionalmente, la presencia de colas largas y eventos de alta latencia en la percepción indica la existencia de episodios de degradación temporal que no pueden explicarse como ruido, sino como un comportamiento estructural asociado a la naturaleza asíncrona y dependiente de los datos del proceso de localización.

Desde una perspectiva de sistema, estos resultados implican que el rendimiento no está limitado por la capacidad de decisión o ejecución, sino por el cierre del bucle de percepción. En consecuencia, el sistema opera en un régimen *perception-bound*, donde las mejoras en el pipeline de control tienen un impacto limitado sobre la latencia total.

Por tanto, cualquier estrategia orientada a reducir la latencia o mejorar la previsibilidad del sistema debe centrarse prioritariamente en el módulo de percepción. Esto incluye la optimización de la frecuencia de actualización o la reducción del coste computacional de los algoritmos de localización.

Finalmente, los resultados ponen de manifiesto que la latencia percibida no solo es mayor que la latencia física, sino que presenta una naturaleza estadística diferente, carac-

terizada por mayor dispersión, asimetría y presencia de valores extremos. Esta diferencia debe ser tenida en cuenta en el diseño y evaluación de sistemas robóticos, ya que afecta directamente a la capacidad de respuesta y a la estabilidad temporal del sistema en condiciones reales de operación.

A. Análisis completo del sistema

1. Carga de datos

In [2]:
```
import json
import pandas as pd
import matplotlib.pyplot as plt

with open("../../data/experiment_analysis.json") as f:
    data = json.load(f)

topics = data["topics"]

df = pd.DataFrame.from_dict(topics, orient="index")
df.reset_index(inplace=True)
df.rename(columns={"index": "topic"}, inplace=True)
```

2. Caracterización del tráfico

2.1. Ancho de banda

In [3]:
```
df["bw_percent"] = df["bandwidth_mb_s"] / df["bandwidth_mb_s"].sum() * 10
df.sort_values(by="bw_percent", ascending=False)[["topic", "bandwidth_mb_
```

Out[3]:

	topic	bandwidth_mb_s	bw_percent
36	/robot/front_rgbd_camera/depth/points	71.088803	70.868156
40	/robot/front_rgbd_camera/depth/image_raw	8.886586	8.859003
50	/robot/front_rgbd_camera/rgb/image_raw	6.665086	6.644399
77	/robot/robot_front_ptz_camera/image_raw	6.665071	6.644384
28	/robot/move_base/GlobalPlanner/potential	3.094075	3.084471
64	/gazebo/link_states	1.972124	1.966003
69	/gazebo/model_states	1.099804	1.096390
37	/robot/front_laser/scan	0.331207	0.330179
60	/robot/move_base/global_costmap/costmap_updates	0.069513	0.069297
1	/robot/map	0.068757	0.068544

In [4]:
```
df_sorted = df.sort_values(by="bw_percent", ascending=False)
df_sorted["cum"] = df_sorted["bw_percent"].cumsum()
n_topics = len(df_sorted)
top_95 = df_sorted[df_sorted["cum"] <= 95]

print("Top topics:", len(top_95))
print("Percentage:", len(top_95) / n_topics * 100)
```

```
Top topics: 4
Percentage: 4.761904761904762
```

2.2. Frecuencia

In [5]:
```
df.sort_values(by="frequency_hz", ascending=False)[["topic", "frequency_h
```

Out[5]:

	topic	frequency_hz
70	/clock	997.216015
64	/gazebo/link_states	997.069490
69	/gazebo/model_states	996.739807
8	/tf	136.928093
58	/robot/joint_states	100.131873
75	/robot/joint_pan_position_controller/state	100.021979
26	/robot/joint_tilt_position_controller/state	99.820506
78	/robot/imu/data_raw	98.684934
15	/robot/imu/rpy/filtered	98.684934
7	/robot/imu/data	98.684934

2.3. Tamaño

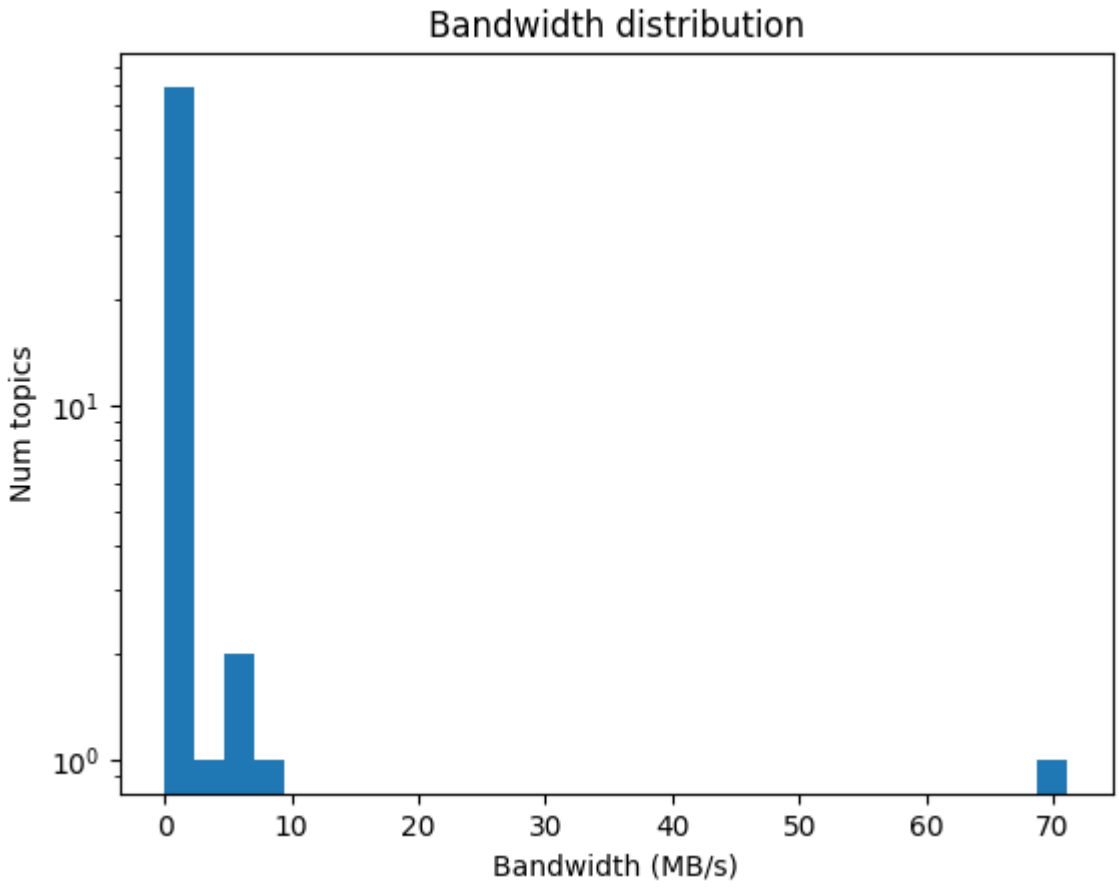
In [6]:
```
df.sort_values(by="avg_size_kb", ascending=False)[["topic","avg_size_kb"]
```

Out[6]:

	topic	avg_size_kb
36	/robot/front_rgbd_camera/depth/points	9600.139648
1	/robot/map	3844.102539
28	/robot/move_base/GlobalPlanner/potential	3844.102539
66	/robot/move_base/global_costmap/costmap	3844.102539
40	/robot/front_rgbd_camera/depth/image_raw	1200.083008
50	/robot/front_rgbd_camera/rgb/image_raw	900.082031
77	/robot/robot_front_ptz_camera/image_raw	900.080078
60	/robot/move_base/global_costmap/costmap_updates	82.687500
45	/robot/move_base/local_costmap/costmap	39.166016
63	/robot/particlecloud	31.696289

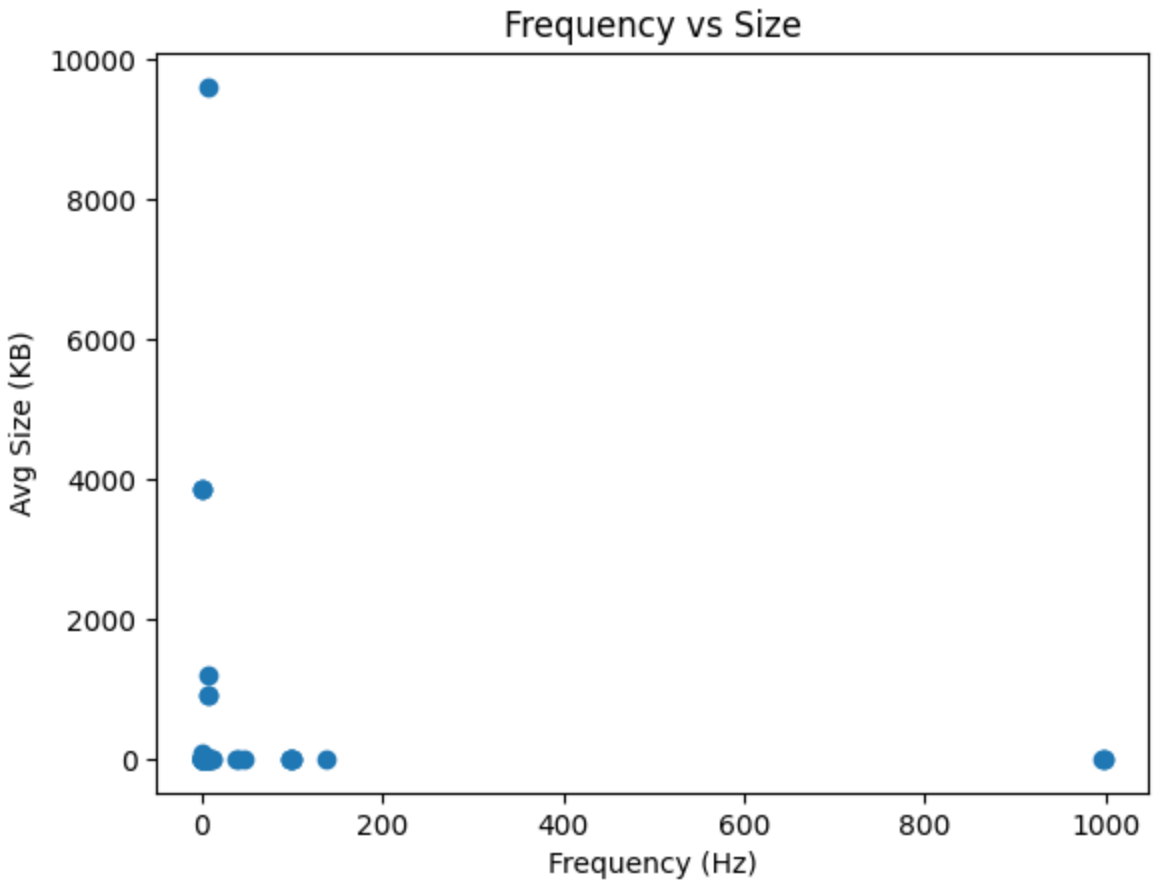
2.4. Distribución global

In [7]:
```
plt.hist(df["bandwidth_mb_s"], bins=30)
plt.xlabel("Bandwidth (MB/s)")
plt.ylabel("Num topics")
plt.title("Bandwidth distribution")
plt.yscale("log")
plt.show()
```



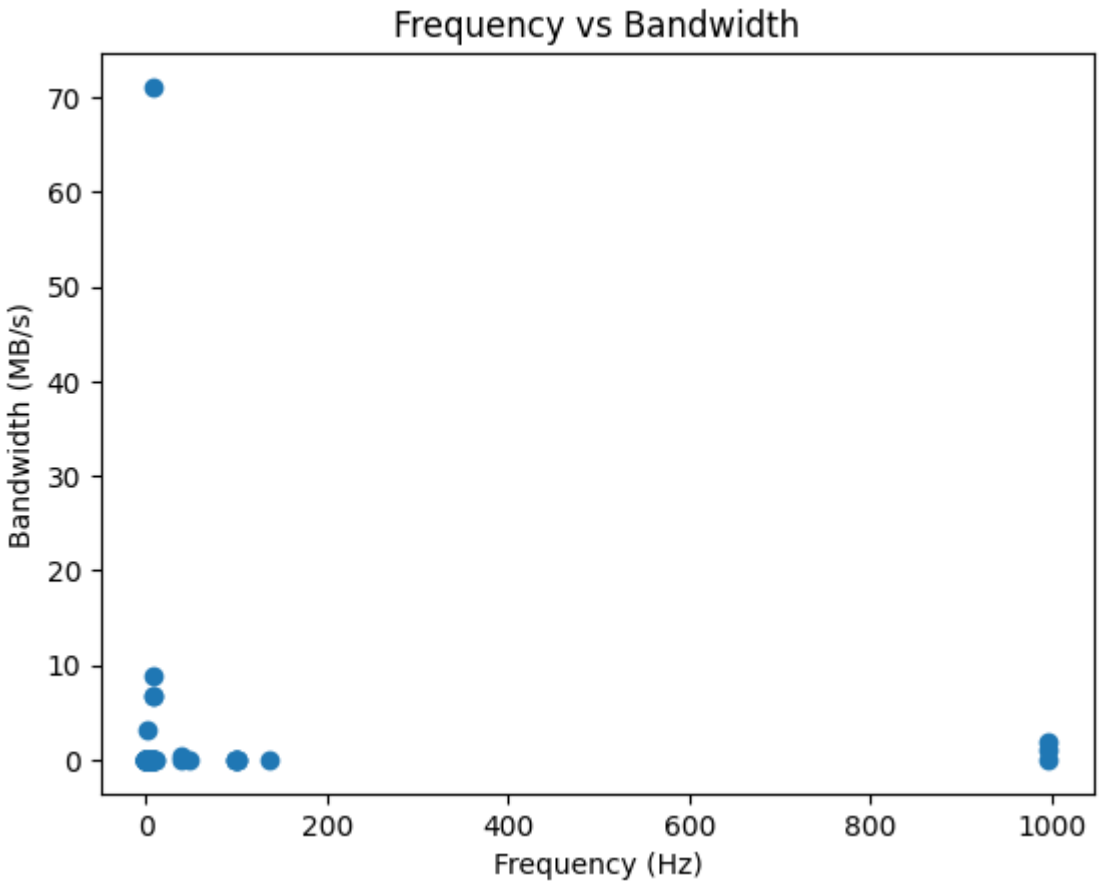
2.5. Relación frecuencia vs tamaño

In [8]:
```
plt.scatter(df["frequency_hz"], df["avg_size_kb"])
plt.xlabel("Frequency (Hz)")
plt.ylabel("Avg Size (KB)")
plt.title("Frequency vs Size")
plt.show()
```



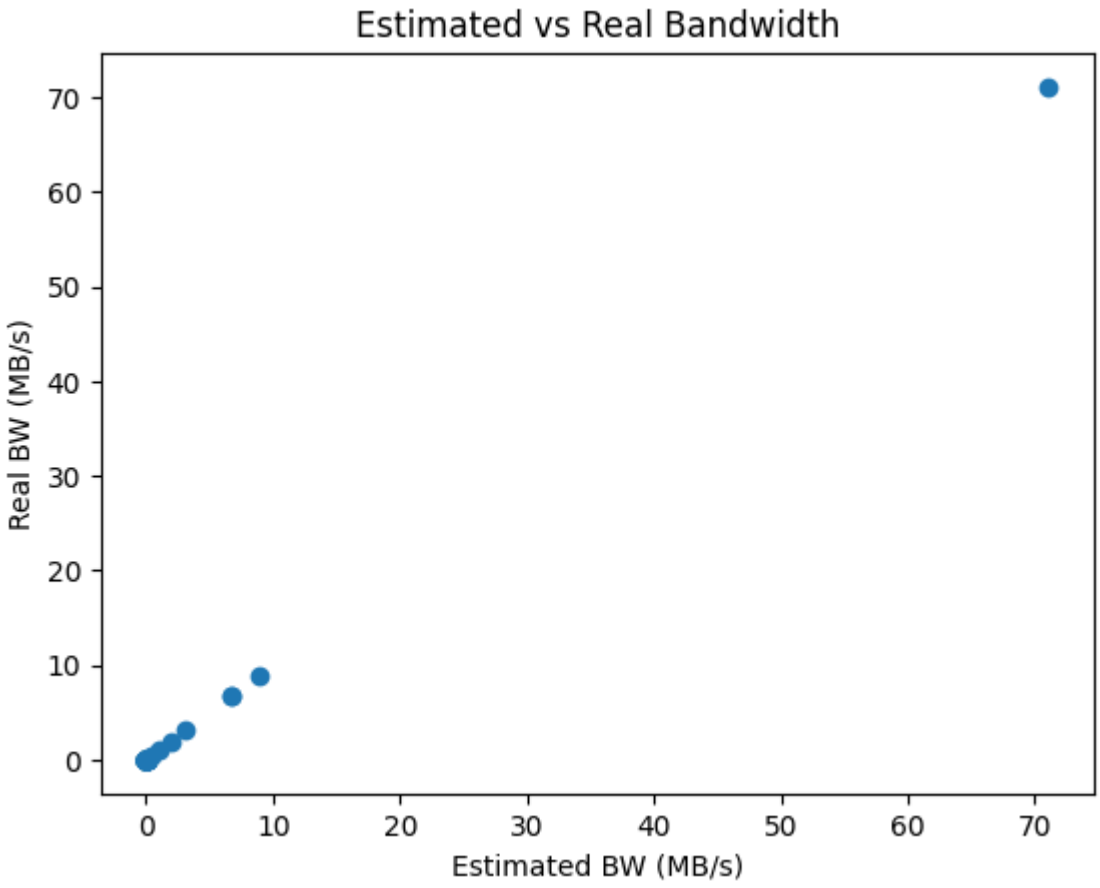
2.6. Relación frecuencia vs bandwidth

In [9]:
```
plt.scatter(df["frequency_hz"], df["bandwidth_mb_s"])
plt.xlabel("Frequency (Hz)")
plt.ylabel("Bandwidth (MB/s)")
plt.title("Frequency vs Bandwidth")
plt.show()
```



2.7. Validación BW ≈ frecuencia × tamaño

In [10]:
```
df["bw_estimated"] = df["frequency_hz"] * df["avg_size_kb"] / 1024

plt.scatter(df["bw_estimated"], df["bandwidth_mb_s"])
plt.xlabel("Estimated BW (MB/s)")
plt.ylabel("Real BW (MB/s)")
plt.title("Estimated vs Real Bandwidth")
plt.show()
```



In [11]:
```
from sklearn.metrics import r2_score

r2 = r2_score(df["bandwidth_mb_s"], df["bw_estimated"])
print("R2:", r2)
```

```
R2: 0.9999999999983807
```

2.8. Conclusiones

El tráfico del sistema presenta una distribución fuertemente concentrada, donde un subconjunto reducido de *topics* acumula la mayor parte del ancho de banda total. Empíricamente, aproximadamente el 95% del tráfico se concentra en torno al X% de los *topics*, confirmando un comportamiento tipo long-tail.

El análisis muestra que el ancho de banda está principalmente determinado por el tamaño de los mensajes y no por la frecuencia de publicación. Esto se valida mediante la fuerte correlación entre el ancho de banda real y el estimado (BW ≈ frecuencia × tamaño).

Los *topics* dominantes corresponden a sensores de percepción (cámaras RGB-D y *point clouds*), lo que indica que el sistema está fuertemente condicionado por la carga de datos sensoriales.

En contraste, la mayoría de los *topics* tienen un impacto marginal en el tráfico global.

3. Selección de flujo crítico

3.1. Filtrado de flujo crítico

In [12]:
```
BW_THRESHOLD = df["bandwidth_mb_s"].quantile(0.9)

critical_topics = df[df["bandwidth_mb_s"] > BW_THRESHOLD].copy().sort_val
```

3.2. Conclusiones

La selección basada en percentil (p90) permite aislar los *topics* que dominan el consumo de red. Estos *topics* representan los principales candidatos para optimización, ya que pequeñas mejoras en ellos tienen un impacto significativo en el sistema global.

Este filtrado reduce el espacio de análisis sin pérdida relevante de información, centrando el estudio en los flujos realmente críticos.

4. Analisis estructural

4.1 Carga grafo

In [13]:
```
with open("../../data/node_topic_graph.json") as f:
    graph = json.load(f)
```

4.2. Dependencias

In [14]:
```
rows = []

for _, row in critical_topics.iterrows():
    topic = row["topic"]
    pubs = graph.get(topic, {}).get("publishers", [])
    subs = graph.get(topic, {}).get("subscribers", [])

    rows.append({
        "topic": topic,
        "bw": row["bandwidth_mb_s"],
        "publishers": pubs,
        "subscribers": subs,
        "n_pub": len(pubs),
        "n_sub": len(subs)
    })

df_dep = pd.DataFrame(rows).sort_values(by="bw", ascending=False)
```

In [15]:
```
df_dep[df_dep["n_sub"] == 0]
```

Out[15]:

topic	bw	publishers	subscribers	n_pub	n_sub

4.3. Clasificación funcional

In [16]:
```
def classify_node(node):
    if any(k in node for k in ["camera", "laser", "points", "image", "dep
        return "perception"
    elif "move_base" in node:
        return "planning"
    elif "amcl" in node:
        return "localization"
    elif "costmap" in node:
        return "mapping"
    elif "cmd_vel" in node:
        return "control"
    else:
        return "system"

df_dep["pub_type"] = df_dep["publishers"].apply(lambda x: [classify_node(
df_dep["sub_type"] = df_dep["subscribers"].apply(lambda x: [classify_node
```

4.4.Clasificación funcional agragada

In [17]:
```
from collections import Counter

def dominant_type(types):
    if not types:
        return "unknown"
    return Counter(types).most_common(1)[0][0]

df_dep["main_type"] = df_dep["pub_type"].apply(dominant_type)
df_dep.groupby("main_type")["bw"].sum().sort_values(ascending=False)
```

Out[17]:
```
main_type
system      96.708680
planning     3.163588
Name: bw, dtype: float64
```

4.5. Conclusiones

El análisis estructural revela que los *topics* de mayor ancho de banda suelen presentar un alto número de suscriptores, amplificando su impacto en la red.

La clasificación funcional muestra que la carga del sistema está dominada por componentes de percepción, mientras que los módulos de planificación y control tienen un impacto significativamente menor en términos de tráfico.

Esto sugiere que el sistema sigue una arquitectura típica de robótica: alta generación de datos en sensores y consumo distribuido en múltiples módulos.

5. Dinámica temporal

5.1. Métricas

In [18]:
```
df_dep["lat_mean_ms"] = df_dep["topic"].apply(lambda t: topics[t].get("la
df_dep["jitter_ms"] = df_dep["topic"].apply(lambda t: topics[t].get("jitt
df_dep["peak_bw"] = df_dep["topic"].apply(lambda t: topics[t].get("peak_b
```

In [19]:
```
df_dep.sort_values(
    by=["lat_mean_ms","bw"],
    ascending=[False, False]
)[["topic","bw","lat_mean_ms"]].head(10)
```

Out[19]:

	topic	bw	lat_mean_ms
3	/robot/robot_front_ptz_camera/image_raw	6.665071	131.855072
2	/robot/front_rgbd_camera/rgb/image_raw	6.665086	108.101449
1	/robot/front_rgbd_camera/depth/image_raw	8.886586	107.681159
0	/robot/front_rgbd_camera/depth/points	71.088803	105.280193
4	/robot/move_base/GlobalPlanner/potential	3.094075	18.733333
7	/robot/front_laser/scan	0.331207	14.233256
8	/robot/move_base/global_costmap/costmap_updates	0.069513	0.489362
5	/gazebo/link_states	1.972124	0.000000
6	/gazebo/model_states	1.099804	0.000000

In [20]:
```
# Trafico explosivo
df_dep["burst_ratio"] = df_dep.apply(
    lambda r: r["peak_bw"] / r["bw"] if r["bw"] > 0 else 0,
    axis=1
)

df_dep.sort_values(by="burst_ratio", ascending=False)[[
    "topic", "bw", "peak_bw", "burst_ratio"
]].head(10)
```

Out[20]:

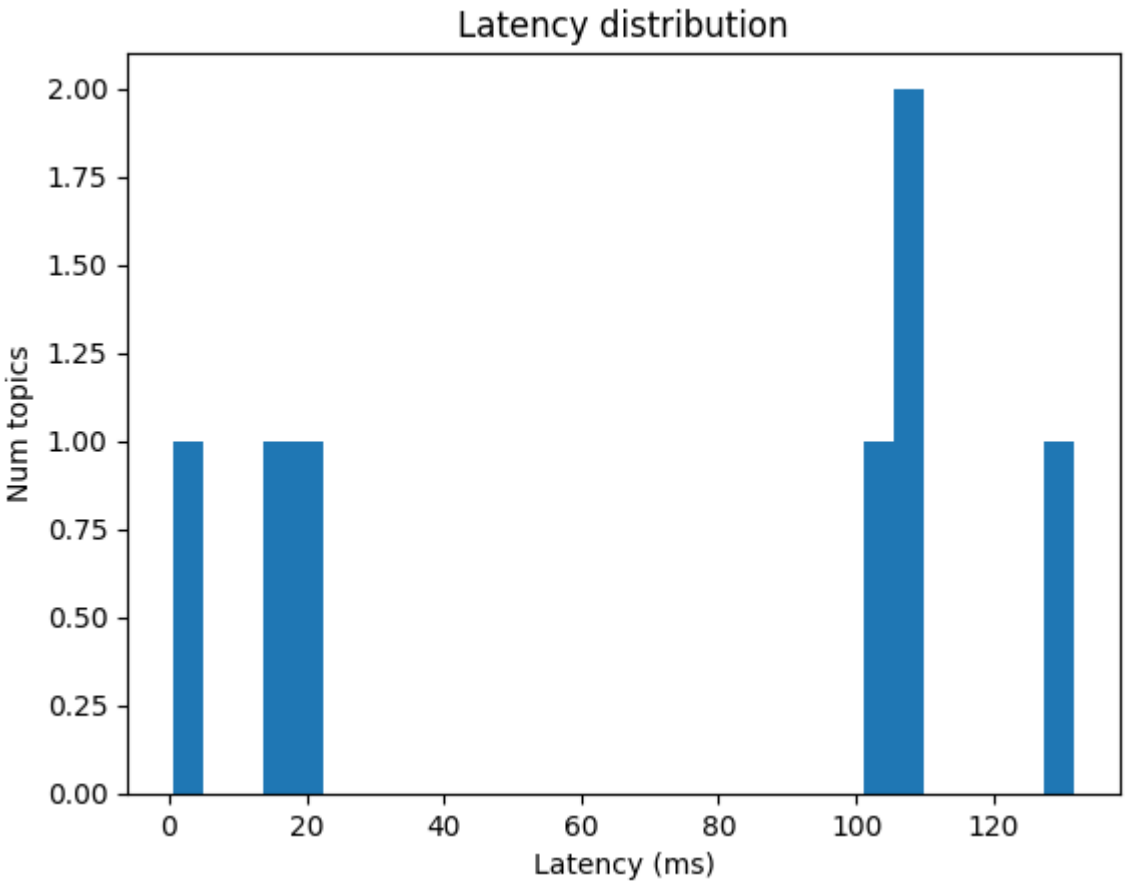
	topic	bw	peak_bw	burst_ratio
7	/robot/front_laser/scan	0.331207	8.317947	25.114075
4	/robot/move_base/GlobalPlanner/potential	3.094075	12.228034	3.952081
2	/robot/front_rgbd_camera/rgb/image_raw	6.665086	13.522867	2.028911
1	/robot/front_rgbd_camera/depth/image_raw	8.886586	16.506423	1.857454
3	/robot/robot_front_ptz_camera/image_raw	6.665071	12.380063	1.857454
0	/robot/front_rgbd_camera/depth/points	71.088803	128.426526	1.806565
8	/robot/move_base/global_costmap/costmap_updates	0.069513	0.080750	1.161646
6	/gazebo/model_states	1.099804	1.103401	1.003271
5	/gazebo/link_states	1.972124	1.977921	1.002939

5.2. Limpiar datos inválidos

In [21]:
```
# no es posible recoger la información de latencia de algunos topics porq
df_dep["lat_available"] = df_dep["lat_mean_ms"] > 0
```

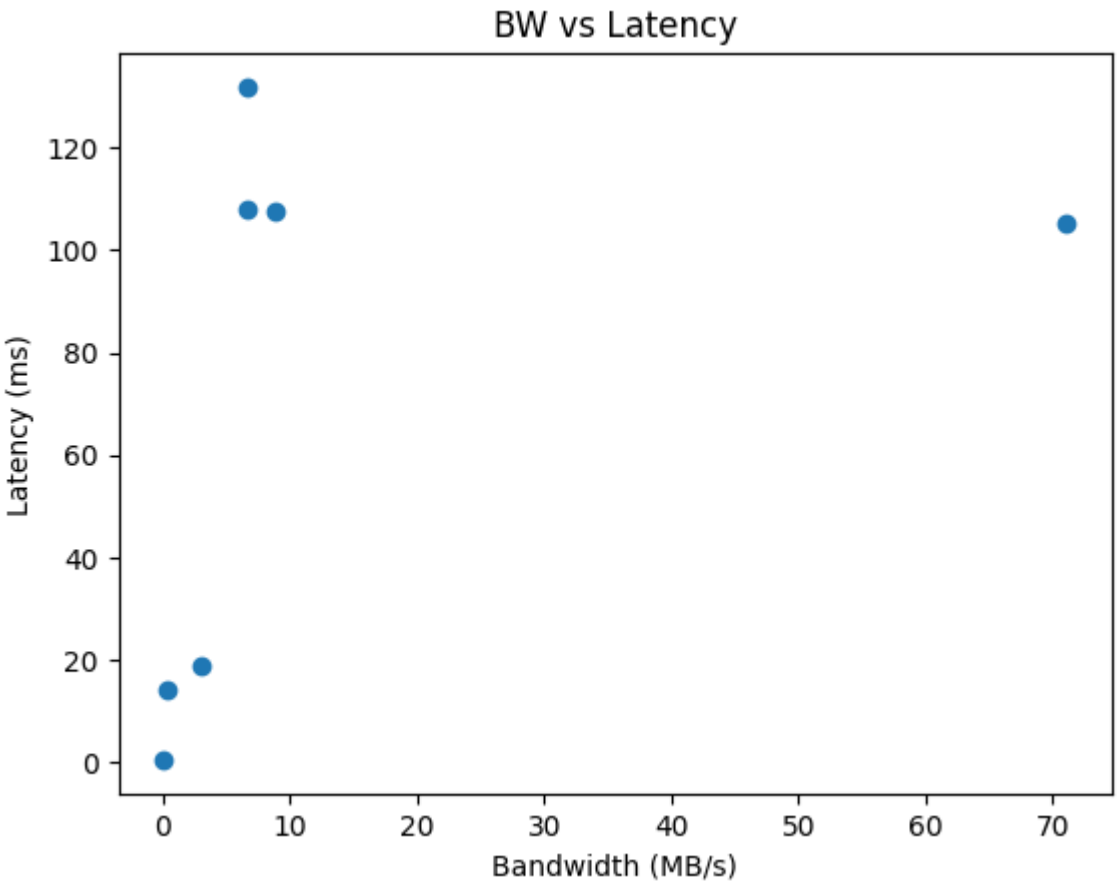
5.3. Distribución de latencia

In [22]:
```
valid = df_dep[df_dep["lat_available"]]
plt.hist(valid["lat_mean_ms"], bins=30)
plt.xlabel("Latency (ms)")
plt.ylabel("Num topics")
plt.title("Latency distribution")
plt.show()
```



5.4. Relación BW vs Latencia

In [23]:
```
valid = df_dep[df_dep["lat_available"]]
plt.scatter(valid["bw"], valid["lat_mean_ms"])
plt.xlabel("Bandwidth (MB/s)")
plt.ylabel("Latency (ms)")
plt.title("BW vs Latency")
plt.show()
```



5.5. Jitter

In [24]:
```
df_dep.sort_values(by="jitter_ms", ascending=False)[["topic","jitter_ms"]]
```

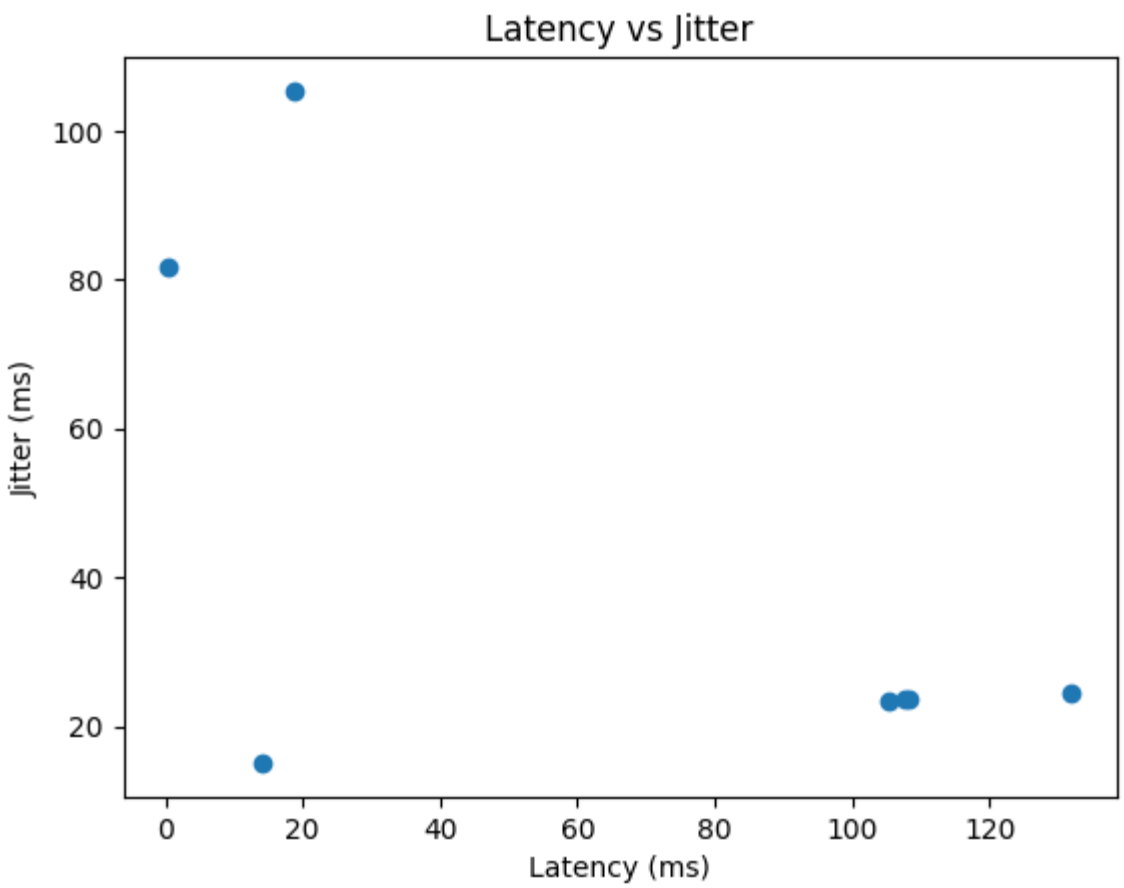
Out[24]:

	topic	jitter_ms
4	/robot/move_base/GlobalPlanner/potential	105.406974
8	/robot/move_base/global_costmap/costmap_updates	81.660682
3	/robot/robot_front_ptz_camera/image_raw	24.367109
1	/robot/front_rgbd_camera/depth/image_raw	23.675922
2	/robot/front_rgbd_camera/rgb/image_raw	23.527657
0	/robot/front_rgbd_camera/depth/points	23.228384
7	/robot/front_laser/scan	15.046288
6	/gazebo/model_states	4.266597
5	/gazebo/link_states	3.714627

5.6. Relación Latencia vs Jitter

In [25]:
```
valid = df_dep[df_dep["lat_available"]]
plt.scatter(valid["lat_mean_ms"], valid["jitter_ms"])
plt.xlabel("Latency (ms)")
plt.ylabel("Jitter (ms)")
plt.title("Latency vs Jitter")
```

```
plt.show()
```



5.7. Correlaciones

In [26]:
```
df_dep[["bw","lat_mean_ms","jitter_ms","n_sub","burst_ratio"]].corr()
```

Out[26]:

	bw	lat_mean_ms	jitter_ms	n_sub	burst_ratio
bw	1.000000	0.454072	-0.133163	-0.235165	-0.173966
lat_mean_ms	0.454072	1.000000	-0.217268	-0.373971	-0.242297
jitter_ms	-0.133163	-0.217268	1.000000	-0.030439	-0.123172
n_sub	-0.235165	-0.373971	-0.030439	1.000000	0.928507
burst_ratio	-0.173966	-0.242297	-0.123172	0.928507	1.000000

In [27]:
```
df_dep[["bw","lat_mean_ms","jitter_ms","n_sub","burst_ratio"]].corr()["la
```

Out[27]:
```
bw             0.454072
lat_mean_ms    1.000000
jitter_ms     -0.217268
n_sub         -0.373971
burst_ratio   -0.242297
Name: lat_mean_ms, dtype: float64
```

5.8. Outliers

In [28]:
```
df_dep[df_dep["lat_mean_ms"] > 500][[
```

```
    "topic","bw","lat_mean_ms","n_sub"
]]
```

Out[28]:

topic	bw	lat_mean_ms	n_sub

In [29]:

```
df_dep[df_dep["burst_ratio"] > 5][[
    "topic","bw","peak_bw","burst_ratio"
]]
```

Out[29]:

	topic	bw	peak_bw	burst_ratio
7	/robot/front_laser/scan	0.331207	8.317947	25.114075

5.9. Cuellos de botella

In [ ]:

```
df_dep[
    (df_dep["lat_mean_ms"] > df_dep["lat_mean_ms"].quantile(0.9)) &
    (df_dep["bw"] > df_dep["bw"].quantile(0.8))
][["topic","bw","lat_mean_ms"]]
```

5.10 Conclusiones

La latencia presenta una distribución heterogénea, con la mayoría de los *topics* en rangos moderados y la presencia de outliers significativos que pueden comprometer el comportamiento del sistema.

No existe una correlación fuerte entre ancho de banda y latencia, lo que indica que los problemas de latencia no dependen únicamente del volumen de datos, sino también de factores como procesamiento, serialización o scheduling.

El jitter muestra variabilidad relevante en ciertos *topics*, lo que puede afectar negativamente a sistemas de control en tiempo real.

El análisis de *burst ratio* evidencia la existencia de tráfico no uniforme en el tiempo, con picos que pueden generar congestión puntual.

En conjunto, el sistema presenta un comportamiento temporal no determinista, con posibles cuellos de botella en *topics* específicos.

6. Impacto final

6.1. Normalización

In [30]:

```
max_bw = df_dep["bw"].max()
valid_lat = df_dep[df_dep["lat_available"]]["lat_mean_ms"]
max_lat = valid_lat.max() if not valid_lat.empty else 0

df_dep["bw_norm"] = df_dep["bw"] / max_bw if max_bw > 0 else 0
df_dep["lat_norm"] = df_dep["lat_mean_ms"] / max_lat if max_lat > 0 else
```

```
df_dep.loc[~df_dep["lat_available"], "lat_norm"] = 0
```

6.2. Clasificación final

In [31]:
```
df_dep["net_impact"] = df_dep["bw"] * df_dep["n_sub"]
df_dep["fanout_ratio"] = df_dep.apply(
    lambda r: r["n_sub"] / r["n_pub"] if r["n_pub"] > 0 else 0,
    axis=1
)
df_dep["fanout_impact"] = df_dep["bw"] * df_dep["fanout_ratio"]
bw_thr = df_dep["bw"].quantile(0.8)
jitter_thr = df_dep["jitter_ms"].quantile(0.9)
def classify(row):
    if not row["lat_available"]:
        return "unknown"
    elif row["bw"] > bw_thr and row["lat_mean_ms"] > 80:
        return "critical"
    elif row["bw"] > bw_thr:
        return "high_bw"
    elif row["jitter_ms"] > jitter_thr:
        return "unstable"
    else:
        return "low_impact"

df_dep["impact_class"] = df_dep.apply(classify, axis=1)
```

6.3. Ranking final

In [32]:
```
df_dep.sort_values(
    by=["impact_class","fanout_impact","net_impact"],
    ascending=[True, False, False]
)[[
    "topic","impact_class","fanout_impact","net_impact","bw","lat_mean_ms
]]
```

Out[32]:

	topic	impact_class	fanout_impact	net_impact	bw	lat_me
0	/robot/front_rgbd_camera/depth/points	critical	71.088803	71.088803	71.088803	105.2
1	/robot/front_rgbd_camera/depth/image_raw	critical	8.886586	8.886586	8.886586	107.6
2	/robot/front_rgbd_camera/rgb/image_raw	low_impact	6.665086	6.665086	6.665086	108.1
3	/robot/robot_front_ptz_camera/image_raw	low_impact	6.665071	6.665071	6.665071	131.8
7	/robot/front_laser/scan	low_impact	1.324826	1.324826	0.331207	14.2
8	/robot/move_base/global_costmap/costmap_updates	low_impact	0.139026	0.139026	0.069513	0.4
5	/gazebo/link_states	unknown	1.972124	1.972124	1.972124	0.0
6	/gazebo/model_states	unknown	1.099804	1.099804	1.099804	0.0
4	/robot/move_base/GlobalPlanner/potential	unstable	3.094075	3.094075	3.094075	18.7

6.4. Descomposición del impacto

In [33]:

```
df_dep["time_impact"] = df_dep["lat_mean_ms"]
df_dep["network_impact"] = df_dep["net_impact"]
df_dep["stability_impact"] = df_dep["jitter_ms"]

df_dep[[
    "topic","network_impact","time_impact","stability_impact"
]].sort_values(by="network_impact", ascending=False)
```

Out[33]:

	topic	network_impact	time_impact	stability_impact
0	/robot/front_rgbd_camera/depth/points	71.088803	105.280193	23.228384
1	/robot/front_rgbd_camera/depth/image_raw	8.886586	107.681159	23.675922
2	/robot/front_rgbd_camera/rgb/image_raw	6.665086	108.101449	23.527657
3	/robot/robot_front_ptz_camera/image_raw	6.665071	131.855072	24.367109
4	/robot/move_base/GlobalPlanner/potential	3.094075	18.733333	105.406974
5	/gazebo/link_states	1.972124	0.000000	3.714627
7	/robot/front_laser/scan	1.324826	14.233256	15.046288
6	/gazebo/model_states	1.099804	0.000000	4.266597
8	/robot/move_base/global_costmap/costmap_updates	0.139026	0.489362	81.660682

In [36]:

```
df_dep.sort_values(
    by=["network_impact","time_impact","stability_impact"],
    ascending=[False, False, False]
)
```

Out[36]:

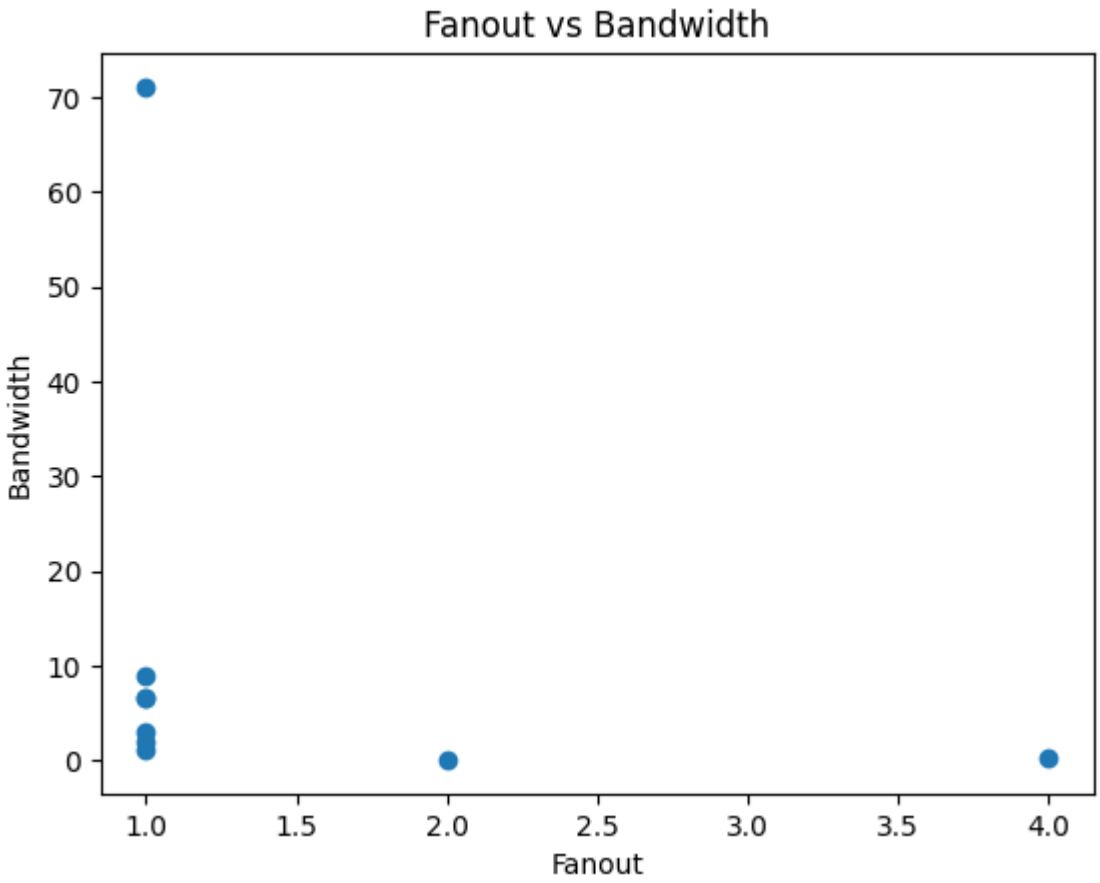
	topic	bw	publishers	subscribers	n_pub	n_sub	pub_t
0	/robot/front_rgbd_camera/depth/points	71.088803	[/gazebo]	[/bag_recorder]	1	1	[syst
1	/robot/front_rgbd_camera/depth/image_raw	8.886586	[/gazebo]	[/bag_recorder]	1	1	[syst
2	/robot/front_rgbd_camera/rgb/image_raw	6.665086	[/gazebo]	[/bag_recorder]	1	1	[syst
3	/robot/robot_front_ptz_camera/image_raw	6.665071	[/gazebo]	[/bag_recorder]	1	1	[syst
4	/robot/move_base/GlobalPlanner/potential	3.094075	[/robot/move_base]	[/bag_recorder]	1	1	[planni
5	/gazebo/link_states	1.972124	[/gazebo]	[/bag_recorder]	1	1	[syst
7	/robot/front_laser/scan	0.331207	[/gazebo]	[/bag_recorder, /robot/amcl, /robot/move_base,...	1	4	[syst
6	/gazebo/model_states	1.099804	[/gazebo]	[/bag_recorder]	1	1	[syst
8	/robot/move_base/global_costmap/costmap_updates	0.069513	[/robot/move_base]	[/bag_recorder, /rviz]	1	2	[planni

9 rows × 23 columns

6.5. Relación fanout vs BW

In [34]:

```
plt.scatter(df_dep["fanout_ratio"], df_dep["bw"])
plt.xlabel("Fanout")
plt.ylabel("Bandwidth")
plt.title("Fanout vs Bandwidth")
plt.show()
```



6.6. Candidatos a offloading

In [37]:
```
df_dep[
    (df_dep["impact_class"] == "critical") &
    (df_dep["fanout_impact"] > df_dep["fanout_impact"].quantile(0.8))
][[
    "topic","bw","lat_mean_ms","fanout_impact"
]]
```

Out[37]:

	topic	bw	lat_mean_ms	fanout_impact
0	/robot/front_rgbd_camera/depth/points	71.088803	105.280193	71.088803
1	/robot/front_rgbd_camera/depth/image_raw	8.886586	107.681159	8.886586

6.7. Conclusiones

El impacto de un *topic* no depende únicamente de su ancho de banda, sino de la combinación de múltiples factores: volumen de datos, latencia y número de suscriptores.

El concepto de *fanout impact* permite identificar *topics* que, aunque no sean los más pesados individualmente, generan una carga significativa al distribuirse a múltiples nodos.

Los *topics* clasificados como críticos combinan alto ancho de banda y alta latencia,

representando los principales cuellos de botella del sistema.

Estos resultados permiten identificar candidatos claros para estrategias de optimización, como compresión, reducción de frecuencia o offloading hacia nodos edge.

7. Conclusiones globales

El sistema analizado presenta una arquitectura fuertemente dominada por flujos de datos de percepción, donde un número reducido de *topics* concentra la mayor parte del tráfico y del impacto en red.

Se ha demostrado que el ancho de banda está principalmente determinado por el tamaño de los mensajes, mientras que la latencia y el jitter dependen de factores adicionales relacionados con procesamiento y distribución.

El análisis estructural y temporal permite identificar *topics* críticos que actúan como cuellos de botella, especialmente aquellos con alto volumen de datos y elevado fanout.

Estos resultados justifican la necesidad de aplicar estrategias de optimización, particularmente mediante offloading hacia arquitecturas edge, donde el procesamiento cercano a la fuente de datos puede reducir tanto la latencia como la carga de red.

En conjunto, el análisis proporciona una base sólida para el diseño de arquitecturas distribuidas eficientes en sistemas robóticos conectados.

B. Análisis de la latencia en arquitectura centralizada

1. Carga de datos

1.1. Lectura datos

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from fastdtw import fastdtw
from scipy.spatial.distance import euclidean
```

In [2]:
```
# Ruta al archivo
file_path = "../../data/experiment_0.json"

df = pd.read_json(file_path)

print("="*70)
print("Primeras 5 filas:\n")
print(df.head(5).to_string(index=False))
print("="*70)
```

```
======================================================================
Primeras 5 filas:

 header_stamp  seq_id  t_rx                                                topic
valid  original_stamp          aux                        source_topic
        2.836     119 2.836 /robot/robotnik_base_control/odom_instrumented
False           2.809 4.684623e-07 /robot/robotnik_base_control/odom
        2.836     120 2.836 /robot/robotnik_base_control/odom_instrumented
False           2.831 4.853336e-07 /robot/robotnik_base_control/odom
        2.870     121 2.870 /robot/robotnik_base_control/odom_instrumented
False           2.852 4.684631e-07 /robot/robotnik_base_control/odom
        2.874     122 2.875 /robot/robotnik_base_control/odom_instrumented
False           2.873 4.853344e-07 /robot/robotnik_base_control/odom
        2.911     123 2.912 /robot/robotnik_base_control/odom_instrumented
False           2.893 4.684639e-07 /robot/robotnik_base_control/odom
======================================================================
```

1.2. Analisis de calidad

In [3]:
```
report = pd.DataFrame({
    "column": df.columns,
    "dtype": df.dtypes.values,
    "nulls": df.isna().sum().values,
    "null_%": (df.isna().mean() * 100).round(2).values,
    "unique_values": df.nunique().values
})

print("\n" + "="*70)
print("DATA QUALITY REPORT")
print("="*70)
print(report.to_string(index=False))
print("="*70)

print(f"\nTotal filas: {len(df)}")
```

```
print(f"Total columnas: {df.shape[1]}")
print(f"Total nulos: {df.isna().sum().sum()}")
```

```
======================================================================
DATA QUALITY REPORT
======================================================================
        column   dtype  nulls  null_%  unique_values
  header_stamp float64      0     0.0          48936
        seq_id   int64      0     0.0          54525
          t_rx float64      0     0.0          47064
         topic     str      0     0.0              5
         valid    bool      0     0.0              2
original_stamp float64      0     0.0          55469
           aux float64      0     0.0          20008
  source_topic     str      0     0.0              5
======================================================================

Total filas: 75987
Total columnas: 8
Total nulos: 0
```

2. Normalizacion

2.1. Unificar timestamps

In [4]:
```
df["t_rx"] = df["t_rx"].astype(float)
df["original_stamp"] = df["original_stamp"].astype(float)
```

In [5]:
```
df = df.sort_values("t_rx").reset_index(drop=True)
```

3. Segmentación por goal

3.1 Crear ventanas temporales

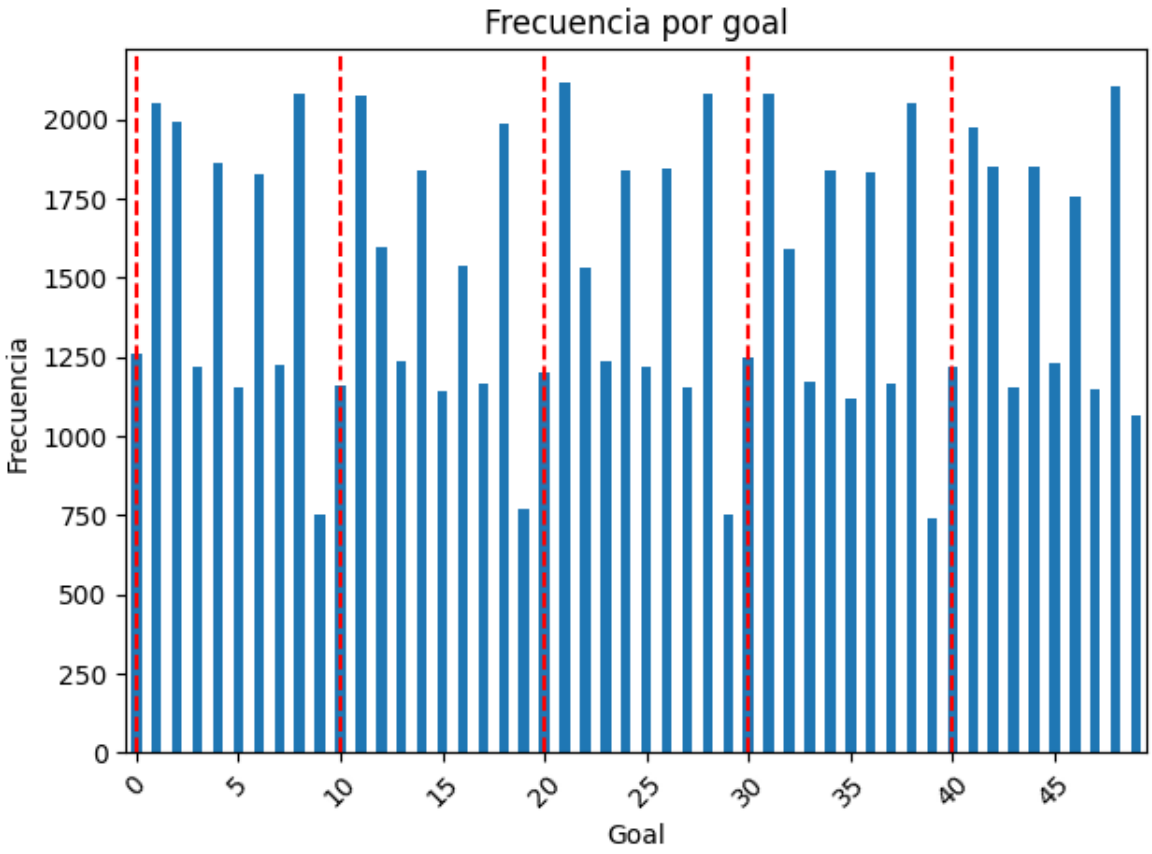
In [6]:
```
segments = []

goal_times = df[df["source_topic"] == "/robot/move_base_simple/goal"]["t_

for i in range(len(goal_times)):
    t_start = goal_times[i]
    t_end = goal_times[i+1] if i < len(goal_times)-1 else df["t_rx"].max(

    segment = df[(df["t_rx"] >= t_start) & (df["t_rx"] < t_end)].copy()
    segment["goal_id"] = i

    segments.append(segment)

df_segmented = pd.concat(segments).reset_index(drop=True)
```

In [7]:
```
print("="*70)
print("Primeras 5 filas:\n")
print(df_segmented.head(5).to_string(index=False))
print("="*70)
```

```
======================================================================
Primeras 5 filas:

 header_stamp  seq_id   t_rx                                                topi
c  valid  original_stamp      aux                       source_topic  goal_
id
       19.526       1 19.526      /robot/move_base_simple/goal_instrumente
d   True           0.000 0.000000      /robot/move_base_simple/goal
0
       19.530     914 19.530 /robot/robotnik_base_control/odom_instrumente
d   True          19.505 0.338791 /robot/robotnik_base_control/odom
0
       19.530     915 19.530 /robot/robotnik_base_control/odom_instrumente
d   True          19.526 0.338791 /robot/robotnik_base_control/odom
0
       19.570     916 19.570 /robot/robotnik_base_control/odom_instrumente
d   True          19.547 0.338791 /robot/robotnik_base_control/odom
0
       19.570     917 19.570 /robot/robotnik_base_control/odom_instrumente
d   True          19.568 0.338802 /robot/robotnik_base_control/odom
0
======================================================================
```

3.2 Verificación

In [8]:
```
counts = df_segmented["goal_id"].value_counts().sort_index()

plt.figure()
counts.plot(kind="bar")

# Añadir líneas verticales cada 10 valores
for i, goal in enumerate(counts.index):
    if goal % 10 == 0:
        plt.axvline(x=i, color="red", linestyle="--", )



plt.title("Frecuencia por goal")
plt.xlabel("Goal")
plt.ylabel("Frecuencia")
step = 5  # muestra 1 de cada 5 etiquetas (ajusta según densidad)
plt.xticks(
    ticks=range(0, len(counts), step),
    labels=counts.index[::step],
    rotation=45
)
plt.tight_layout()

plt.show()
```

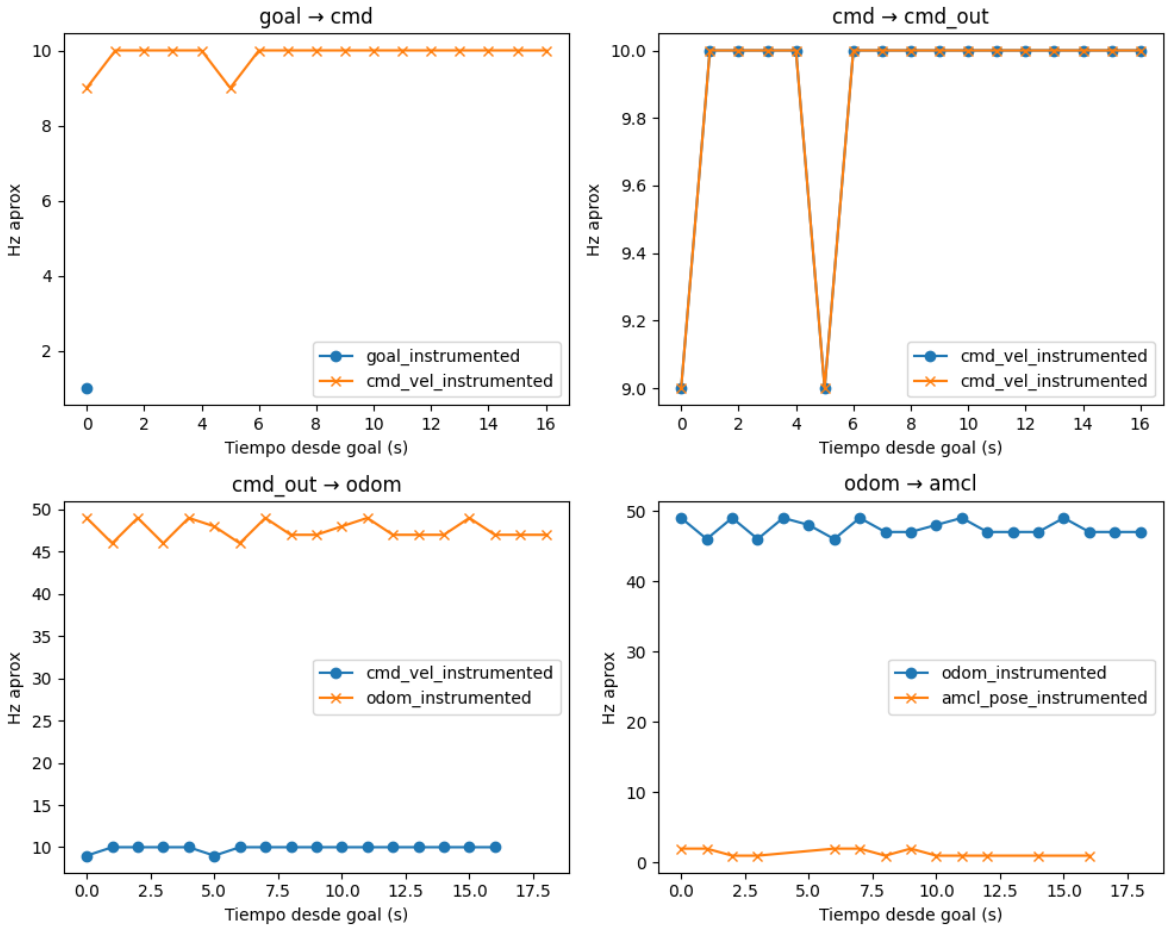


4. Detección de eventos

4.1. Nearest + DTW matching

In [9]:
```
gid = 0

segment = df_segmented[df_segmented["goal_id"] == gid]

# Normalización temporal
t0 = segment[segment["topic"] == "/robot/move_base_simple/goal_instrument

def get_freq(segment, topic, t0):
    df = segment[segment["topic"] == topic].copy()
    df["t_rel"] = df["t_rx"] - t0
    df["sec"] = df["t_rel"].astype(int)
    return df.groupby("sec").size()

pairs = [
    ("/robot/move_base_simple/goal_instrumented", "/robot/move_base/cmd_v
    ("/robot/move_base/cmd_vel_instrumented", "/robot/robotnik_base_contr
    ("/robot/robotnik_base_control/cmd_vel_instrumented", "/robot/robotni
    ("/robot/robotnik_base_control/odom_instrumented", "/robot/amcl_pose_
]

plt.figure(figsize=(10, 8))

for i, (t1, t2, title) in enumerate(pairs, 1):
    f1 = get_freq(segment, t1, t0)
    f2 = get_freq(segment, t2, t0)
```

```
    plt.subplot(2, 2, i)
    plt.plot(f1.index, f1.values, marker='o', label=t1.split("/")[-1])
    plt.plot(f2.index, f2.values, marker='x', label=t2.split("/")[-1])

    plt.title(title)
    plt.xlabel("Tiempo desde goal (s)")
    plt.ylabel("Hz aprox")
    plt.legend()

plt.tight_layout()
plt.show()
```



In [10]:

```
events = []
total_goals = 0
discarded_goals = 0

for gid, segment in df_segmented.groupby("goal_id"):
    total_goals += 1

    # --- GOAL ---
    goal = segment[segment["topic"] == "/robot/move_base_simple/goal_inst
    if goal.empty:
        discarded_goals += 1
        continue
    t_goal = goal["t_rx"].iloc[0]

    # --- CMD ---
    cmd = segment[
        (segment["topic"] == "/robot/move_base/cmd_vel_instrumented") &
        (segment["t_rx"] > t_goal)
```

```
    ]
    if cmd.empty:
        discarded_goals += 1
        continue
    t_cmd = cmd["t_rx"].iloc[0]

    # --- CMD_OUT ---
    cmd_out = segment[
        (segment["topic"] == "/robot/robotnik_base_control/cmd_vel_instru
        (segment["t_rx"] > t_cmd)
    ]
    if cmd_out.empty:
        discarded_goals += 1
        continue
    t_cmd_out = cmd_out["t_rx"].iloc[0]

    # ==========================
    # DTW: CMD_OUT → ODOM
    # ==========================
    cmd_out_times = np.sort(
        segment[
            (segment["topic"] == "/robot/robotnik_base_control/cmd_vel_in
            (segment["t_rx"] > t_cmd)
        ]["t_rx"].values
    )

    odom_times = np.sort(
        segment[
            (segment["topic"] == "/robot/robotnik_base_control/odom_instr
            (segment["t_rx"] > t_cmd_out)
        ]["t_rx"].values
    )

    if len(cmd_out_times) == 0 or len(odom_times) == 0:
        discarded_goals += 1
        continue

    _, path = fastdtw(
        cmd_out_times.reshape(-1,1),
        odom_times.reshape(-1,1),
        dist=euclidean
    )

    idx_cmd_out = np.argmin(np.abs(cmd_out_times - t_cmd_out))

    t_odom = None
    for i, j in path:
        if i == idx_cmd_out:
            t_odom = odom_times[j]
            break

    if t_odom is None or t_odom <= t_cmd_out:
        discarded_goals += 1
        continue

    # ==========================
    # DTW: ODOM → AMCL
    # ==========================
    odom_times = np.sort(
        segment[
```

```
            (segment["topic"] == "/robot/robotnik_base_control/odom_instr
            (segment["t_rx"] > t_cmd_out)
        ]["t_rx"].values
    )

    amcl_times = np.sort(
        segment[
            (segment["topic"] == "/robot/amcl_pose_instrumented") &
            (segment["t_rx"] > t_odom)
        ]["t_rx"].values
    )

    if len(amcl_times) == 0:
        discarded_goals += 1
        continue

    _, path = fastdtw(
        odom_times.reshape(-1,1),
        amcl_times.reshape(-1,1),
        dist=euclidean
    )

    idx_odom = np.argmin(np.abs(odom_times - t_odom))

    t_amcl = None
    for i, j in path:
        if i == idx_odom:
            t_amcl = amcl_times[j]
            break

    if t_amcl is None or t_amcl <= t_odom:
        discarded_goals += 1
        continue

    # --- SAVE ---
    events.append({
        "goal_id": gid,
        "t_goal": t_goal,
        "t_cmd": t_cmd,
        "t_cmd_out": t_cmd_out,
        "t_odom": t_odom,
        "t_amcl": t_amcl
    })

df_events = pd.DataFrame(events)

print("="*70)
print(f"Total goals: {total_goals}")
print(f"Descartados: {discarded_goals}")
print(f"% descartados: {discarded_goals / total_goals * 100:.2f}%")
print("="*70)
```

```
======================================================================
Total goals: 50
Descartados: 0
% descartados: 0.00%
======================================================================
```

In [11]:

```
invalid_order = df_events[
    ~(
```

```
        (df_events["t_goal"] <= df_events["t_cmd"]) &
        (df_events["t_cmd"] <= df_events["t_cmd_out"]) &
        (df_events["t_cmd_out"] <= df_events["t_odom"]) &
        (df_events["t_odom"] <= df_events["t_amcl"])
    )
]
print("="*70)
print(f"Eventos con orden temporal inválido: {len(invalid_order)}")
print("-"*70)
print(f"Filas totales: {len(df_events)}")
print(f"Columnas: {df_events.shape[1]}")
print("-"*70)
print("Primeras 5 filas:\n")
print(df_events.head(5).to_string(index=False))
print("-"*70)
print("="*70)
```

```
======================================================================
Eventos con orden temporal inválido: 0
----------------------------------------------------------------------
Filas totales: 50
Columnas: 6
----------------------------------------------------------------------
Primeras 5 filas:

 goal_id  t_goal   t_cmd  t_cmd_out  t_odom  t_amcl
       0  19.526  19.690     19.787  19.791  19.977
       1  38.480  38.652     38.657  38.658  40.362
       2  69.168  69.344     69.439  69.480  69.844
       3  99.054  99.201     99.297  99.339  99.876
       4 117.478 117.633    117.733 117.741 118.439
----------------------------------------------------------------------
======================================================================
```

4.2. Normalización de timestamps

In [12]:
```
time_cols = ["t_cmd", "t_cmd_out", "t_odom", "t_amcl", "t_goal"]

t0 = df_events["t_goal"].copy()

for col in time_cols:
    df_events[col] = df_events[col] - t0
```

In [13]:
```
print("="*70)
print(f"Filas totales: {len(df_events)}")
print(f"Columnas: {df_events.shape[1]}")
print("-"*70)
print("Primeras 5 filas:\n")
print(df_events.head(5).to_string(index=False))
print("-"*70)
print("="*70)
```

```
======================================================================
Filas totales: 50
Columnas: 6
----------------------------------------------------------------------
Primeras 5 filas:

 goal_id  t_goal  t_cmd  t_cmd_out  t_odom  t_amcl
       0     0.0  0.164      0.261   0.265   0.451
       1     0.0  0.172      0.177   0.178   1.882
       2     0.0  0.176      0.271   0.312   0.676
       3     0.0  0.147      0.243   0.285   0.822
       4     0.0  0.155      0.255   0.263   0.961
----------------------------------------------------------------------
======================================================================
```

5. Calculo de latencias

5.1 Definir métricas

In [14]:
```
df_events["L_plan_control"] = df_events["t_cmd"] - df_events["t_goal"]
df_events["L_mux"]          = df_events["t_cmd_out"] - df_events["t_cmd"]
df_events["L_exec"]         = df_events["t_odom"] - df_events["t_cmd_out"
df_events["L_local"]        = df_events["t_amcl"] - df_events["t_odom"]

df_events["L_e2e_odom"]          = df_events["t_odom"] - df_events["t_goa

df_events["L_e2e_amcl"]     = df_events["t_amcl"] - df_events["t_goal"]
```

In [15]:
```
latency_cols = ["L_plan_control", "L_mux", "L_exec", "L_local", "L_e2e_od

print("="*70)
print(f"Filas totales: {len(df_events)}")
print(f"Columnas: {df_events.shape[1]}")
print("-"*70)
print("Primeras 5 filas:\n")
print(df_events[latency_cols].head(5).to_string(index=False))
print("-"*70)
print("="*70)
```

```
======================================================================
Filas totales: 50
Columnas: 12
----------------------------------------------------------------------
Primeras 5 filas:

 L_plan_control  L_mux  L_exec  L_local  L_e2e_odom  L_e2e_amcl
          0.164  0.097   0.004    0.186       0.265       0.451
          0.172  0.005   0.001    1.704       0.178       1.882
          0.176  0.095   0.041    0.364       0.312       0.676
          0.147  0.096   0.042    0.537       0.285       0.822
          0.155  0.100   0.008    0.698       0.263       0.961
----------------------------------------------------------------------
======================================================================
```

6. Validación y limpieza

In [16]:
```
# Comprobacion de que no hay overlap o intervalos mal definidos
lat_sum = (
    df_events["L_plan_control"] +
    df_events["L_mux"] +
    df_events["L_exec"] +
    df_events["L_local"]
)

error = lat_sum - df_events["L_e2e_amcl"]

print("="*70)
print("Error absoluto medio:", error.abs().mean())
print("Error máximo:", error.abs().max())

tolerance = 1e-6

inconsistent = df_events[error.abs() > tolerance]

print(f"Eventos inconsistentes: {len(inconsistent)}")
print("="*70)
```

```
======================================================================
Error absoluto medio: 0.0
Error máximo: 0.0
Eventos inconsistentes: 0
======================================================================
```

In [17]:
```
# Todas las latencias deben ser >= 0
cols = ["L_plan_control","L_mux","L_exec","L_local","L_e2e_odom","L_e2e_a
print("="*70)
print((df_events[cols] >= 0).all())
print("="*70)
```

```
======================================================================
L_plan_control    True
L_mux             True
L_exec            True
L_local           True
L_e2e_odom        True
L_e2e_amcl        True
dtype: bool
======================================================================
```

7. Análisis estadístico

7.1 Métricas básicas

In [18]:
```
cols = ["L_plan_control","L_mux","L_exec","L_local","L_e2e_odom","L_e2e_a

stats = df_events[cols].agg(["mean","std","min","max","median"])
print("="*70)
print(stats)
print("="*70)
```

```
======================================================================
        L_plan_control     L_mux    L_exec   L_local  L_e2e_odom  L_e2e_am
cl
mean          0.160160  0.069280  0.020040  0.468420    0.249480    0.7179
00
std           0.014345  0.044558  0.012691  0.350244    0.053093    0.3457
31
min           0.132000  0.001000  0.001000  0.004000    0.155000    0.2790
00
max           0.194000  0.107000  0.042000  1.704000    0.328000    1.8820
00
median        0.160500  0.099000  0.020000  0.418000    0.267000    0.6570
00
======================================================================
```

7.2 Percentiles

In [19]:
```
percentiles = df_events[cols].quantile([0.5, 0.9, 0.95, 0.99])
print("="*70)
print(percentiles)
print("="*70)
```

```
======================================================================
      L_plan_control    L_mux   L_exec  L_local  L_e2e_odom  L_e2e_amcl
0.50         0.16050  0.09900  0.02000  0.41800     0.26700     0.65700
0.90         0.17840  0.10110  0.03620  0.74990     0.30410     1.01510
0.95         0.18520  0.10310  0.04010  1.29510     0.31310     1.53155
0.99         0.19253  0.10553  0.04151  1.52809     0.32212     1.74872
======================================================================
```

7.3 Variabilidad relativa

In [20]:
```
cv = df_events[cols].std() / df_events[cols].mean()
print("="*70)
print(cv)
print("="*70)
```

```
======================================================================
L_plan_control    0.089565
L_mux             0.643154
L_exec            0.633280
L_local           0.747714
L_e2e_odom        0.212813
L_e2e_amcl        0.481587
dtype: float64
======================================================================
```

7.4 Contribución por etapa

In [21]:
```
contrib = df_events[["L_plan_control","L_mux","L_exec","L_local"]].mean()
contrib_amcl = contrib / df_events["L_e2e_amcl"].mean()
print("="*70)
print(contrib_amcl)
print("="*70)
```

```
======================================================================
L_plan_control    0.223095
L_mux             0.096504
L_exec            0.027915
L_local           0.652486
dtype: float64
======================================================================
```

7.5 Detección de outliers por percentil (P99)

In [22]:
```
cols = ["L_plan_control","L_mux","L_exec","L_local","L_e2e_odom","L_e2e_a

results = {}

for col in cols:
    p99 = df_events[col].quantile(0.99)

    outliers = df_events[df_events[col] > p99]

    results[col] = {
        "p99": p99,
        "n_outliers": len(outliers),
        "perc_outliers": len(outliers) / len(df_events) * 100
    }

# Mostrar resultados
print("="*70)
for col, res in results.items():
    print(f"\n--- {col} ---")
    print(f"P99: {res['p99']:.4f}")
    print(f"Nº outliers: {res['n_outliers']}")
    print(f"% outliers: {res['perc_outliers']:.2f}%")
print("="*70)
```

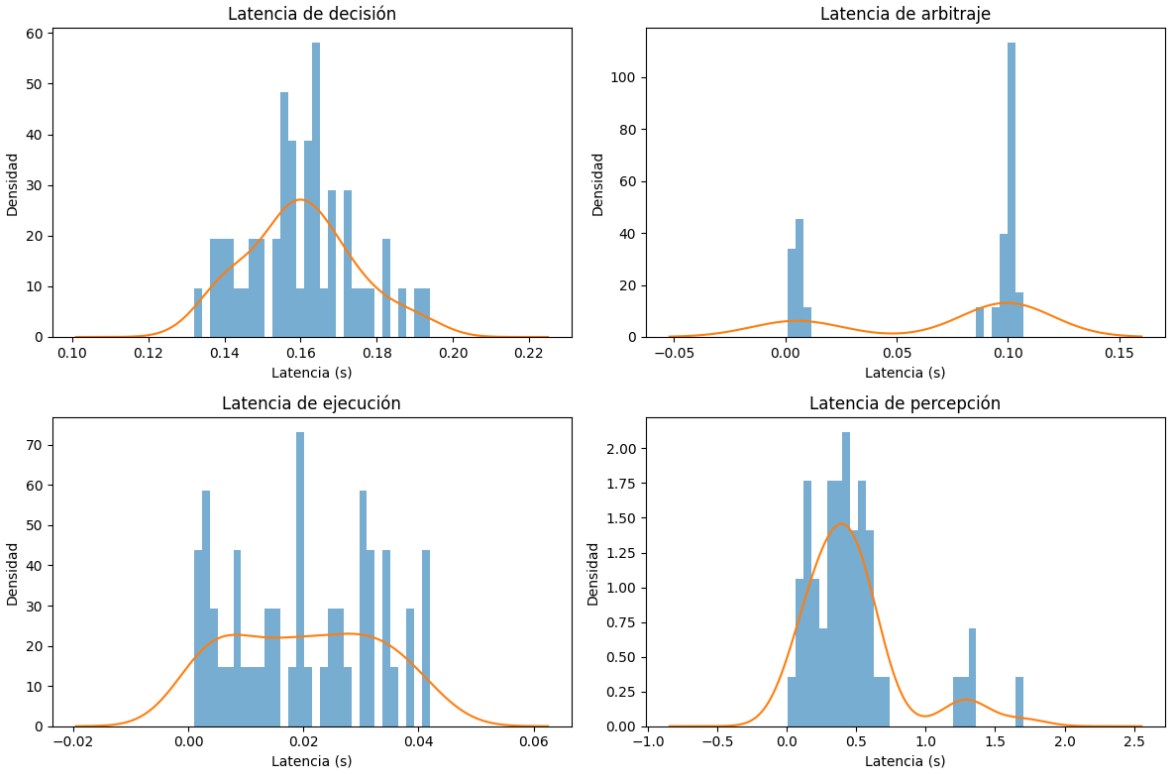
```
======================================================================

--- L_plan_control ---
P99: 0.1925
Nº outliers: 1
% outliers: 2.00%

--- L_mux ---
P99: 0.1055
Nº outliers: 1
% outliers: 2.00%

--- L_exec ---
P99: 0.0415
Nº outliers: 1
% outliers: 2.00%

--- L_local ---
P99: 1.5281
Nº outliers: 1
% outliers: 2.00%

--- L_e2e_odom ---
P99: 0.3221
Nº outliers: 1
% outliers: 2.00%

--- L_e2e_amcl ---
P99: 1.7487
Nº outliers: 1
% outliers: 2.00%
======================================================================
```

8. Visualización

8.1 Histogramas

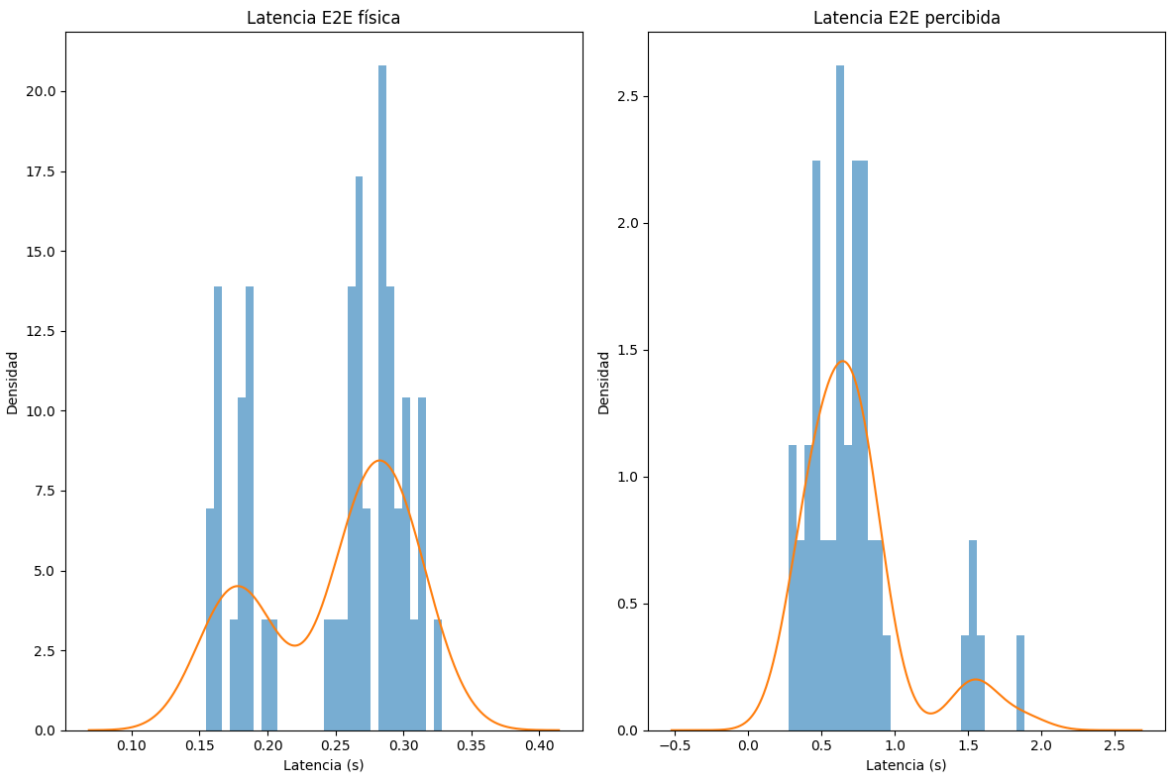
In [23]:
```
cols = ["L_plan_control","L_mux","L_exec","L_local"]

labels = {
    "L_plan_control": "Latencia de decisión",
    "L_mux": "Latencia de arbitraje",
    "L_exec": "Latencia de ejecución",
    "L_local": "Latencia de percepción"
}

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for i, col in enumerate(cols):
    ax = axes[i]

    # Histograma
    df_events[col].plot(kind="hist", bins=30, density=True, ax=ax, alpha=

    # KDE
    df_events[col].plot(kind="kde", ax=ax)
```

```
    ax.set_title(labels[col])
    ax.set_xlabel("Latencia (s)")
    ax.set_ylabel("Densidad")

plt.tight_layout()
plt.show()
```



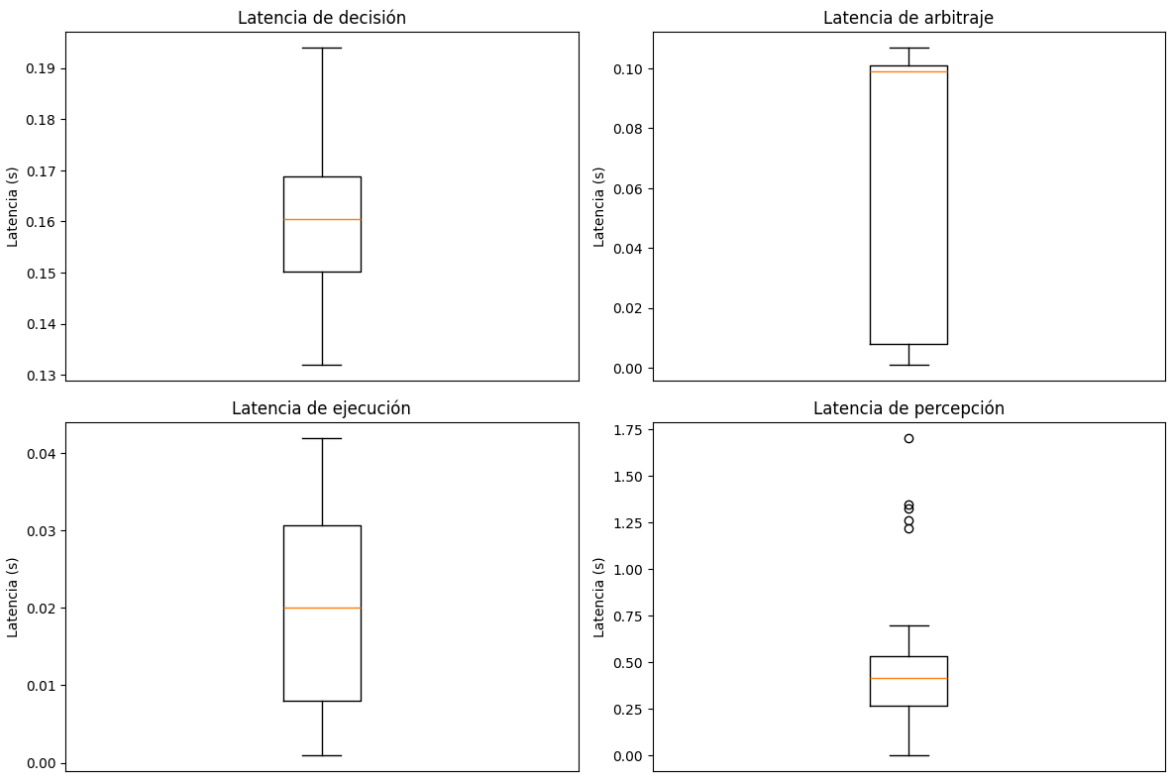
In [24]:

```
cols = ["L_e2e_odom","L_e2e_amcl"]

labels = {
    "L_e2e_odom": "Latencia E2E física",
    "L_e2e_amcl": "Latencia E2E percibida"
}

fig, axes = plt.subplots(1, 2, figsize=(12, 8))
axes = axes.flatten()

for i, col in enumerate(cols):
    ax = axes[i]

    # Histograma
    df_events[col].plot(kind="hist", bins=30, density=True, ax=ax, alpha=

    # KDE
    df_events[col].plot(kind="kde", ax=ax)

    ax.set_title(labels[col])
    ax.set_xlabel("Latencia (s)")
    ax.set_ylabel("Densidad")

plt.tight_layout()
plt.show()
```



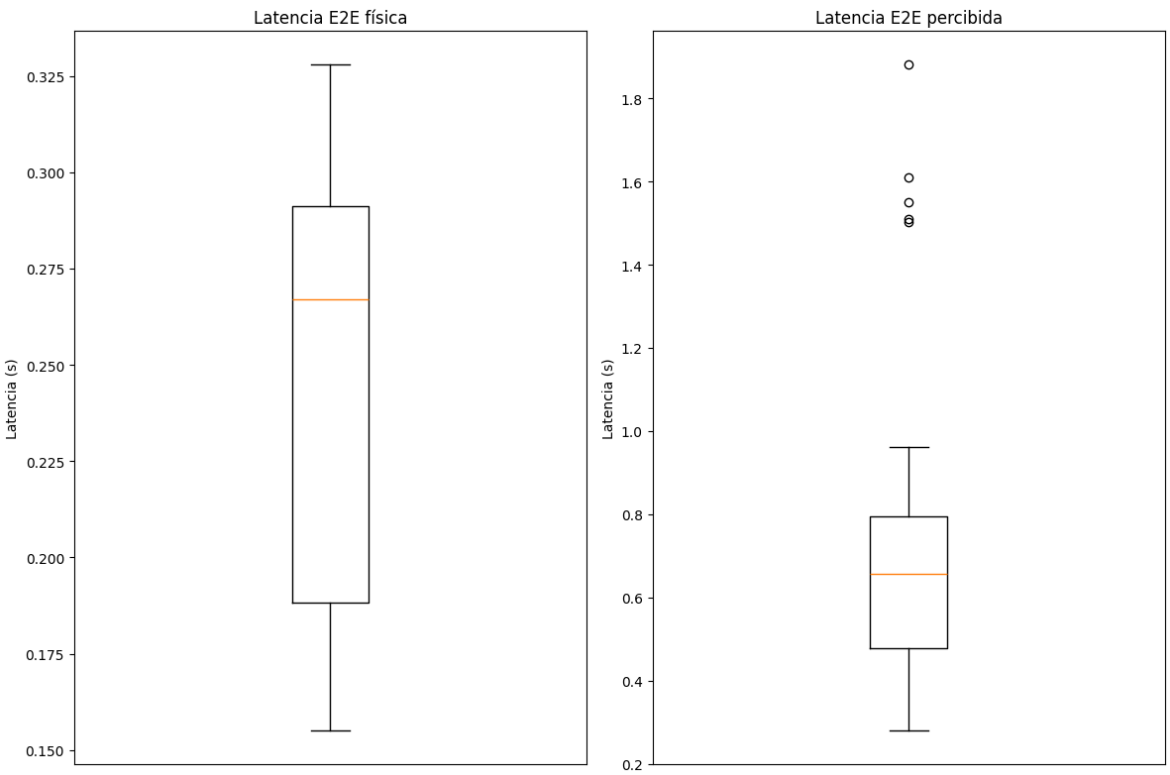
8.2. Boxplot

In [25]:
```
cols = ["L_plan_control","L_mux","L_exec","L_local"]

labels = {
    "L_plan_control": "Latencia de decisión",
    "L_mux": "Latencia de arbitraje",
    "L_exec": "Latencia de ejecución",
    "L_local": "Latencia de percepción"
}

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for i, col in enumerate(cols):
    ax = axes[i]
    ax.boxplot(df_events[col].dropna())
    ax.set_title(labels[col])
    ax.set_ylabel("Latencia (s)")
    ax.set_xticks([])

plt.tight_layout()
plt.show()
```



In [26]:

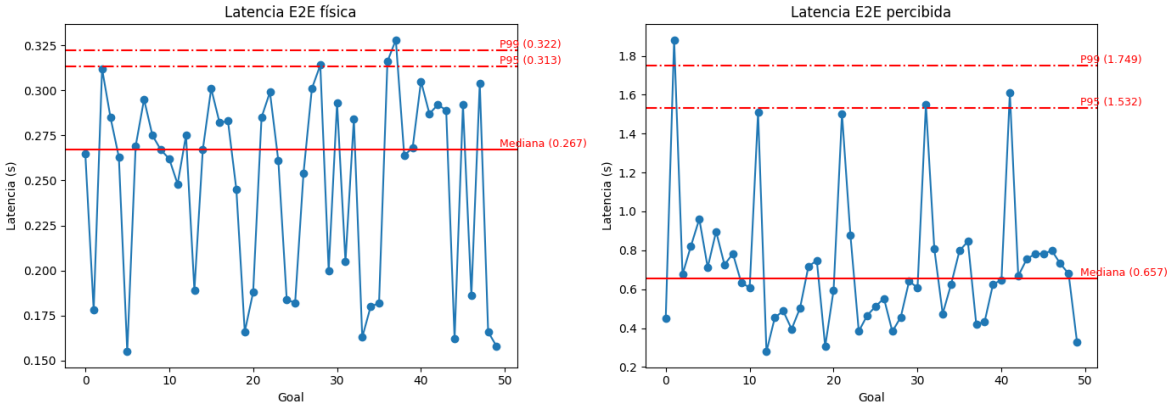
```
cols = ["L_e2e_odom","L_e2e_amcl"]

labels = {
    "L_e2e_odom": "Latencia E2E física",
    "L_e2e_amcl": "Latencia E2E percibida"
}

fig, axes = plt.subplots(1, 2, figsize=(12, 8))
axes = axes.flatten()

for i, col in enumerate(cols):
    ax = axes[i]
    ax.boxplot(df_events[col].dropna())
    ax.set_title(labels[col])
    ax.set_ylabel("Latencia (s)")
    ax.set_xticks([])

plt.tight_layout()
plt.show()
```



8.3. Latencia por goal

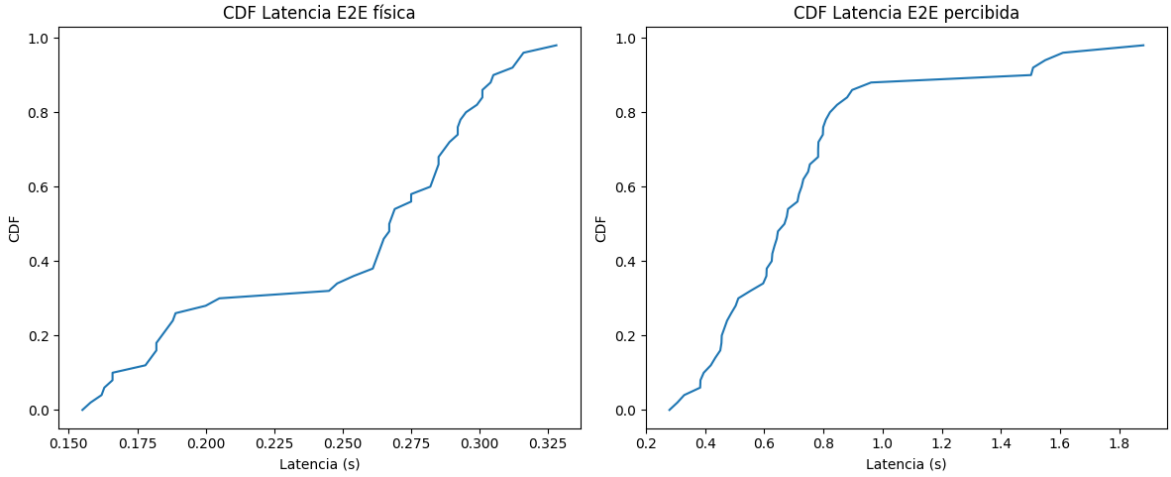
In [27]:
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

cols = ["L_e2e_odom", "L_e2e_amcl"]
titles = ["Latencia E2E física", "Latencia E2E percibida"]

for ax, col, title in zip(axes, cols, titles):

    x = df_events["goal_id"]
    y = df_events[col]

    # métricas
    median = y.median()
    p95 = y.quantile(0.95)
    p99 = y.quantile(0.99)

    ax.plot(x, y, marker='o')

    # función local para cada subplot
    def hline(ax, y_val, text, style):
        ax.axhline(y_val, color='red', linestyle=style)
        ax.text(
            x.max(), y_val,
            f" {text} ({y_val:.3f})",
            color='red',
            fontsize=9,
            verticalalignment='bottom'
        )

    # líneas
    hline(ax, median, "Mediana", '-')
    hline(ax, p95, "P95", '-.')
    hline(ax, p99, "P99", '-.')
```

```
    ax.set_xlabel("Goal")
    ax.set_ylabel("Latencia (s)")
    ax.set_title(title)

plt.tight_layout()
plt.show()
```



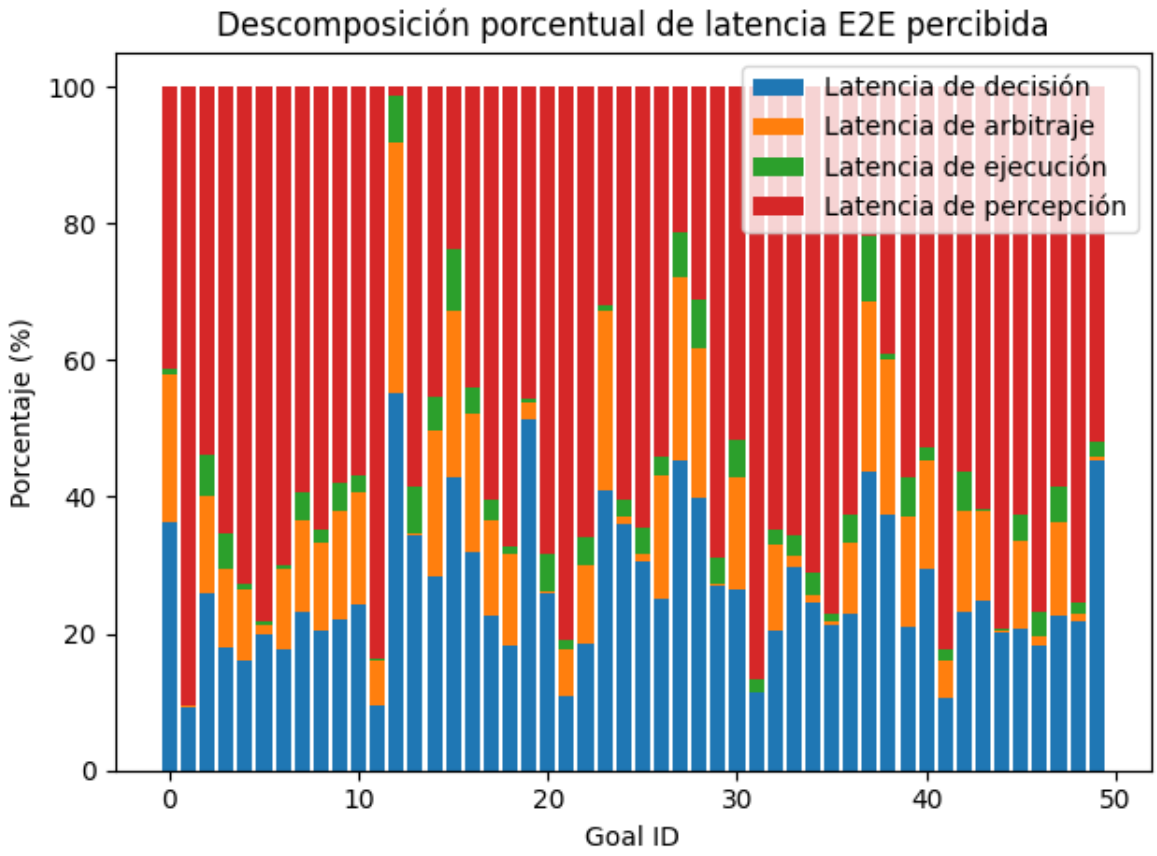
8.4. CDF

In [28]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

cols = ["L_e2e_odom", "L_e2e_amcl"]
titles = ["CDF Latencia E2E física", "CDF Latencia E2E percibida"]


for ax, col, title in zip(axes, cols, titles):

    sorted_lat = np.sort(df_events[col])
    cdf = np.arange(len(sorted_lat)) / len(sorted_lat)

    ax.plot(sorted_lat, cdf)
    ax.set_xlabel("Latencia (s)")
    ax.set_ylabel("CDF")
    ax.set_title(title)

plt.tight_layout()
plt.show()
```



Grafica de barras apiladas normalizadas (100%)

In [29]:
```
cols = ["L_plan_control", "L_mux", "L_exec", "L_local"]

labels = {
    "L_plan_control": "Latencia de decisión",
    "L_mux": "Latencia de arbitraje",
    "L_exec": "Latencia de ejecución",
    "L_local": "Latencia de percepción"
}

total = "L_e2e_amcl"

df_plot = df_events.copy()

# Normalización a porcentaje
for col in cols:
    df_plot[col] = df_plot[col] / df_plot[total] * 100

plt.figure()

bottom = None

for col in cols:
    plt.bar(
        df_plot["goal_id"],
        df_plot[col],
        bottom=bottom,
        label=labels[col]
    )

    if bottom is None:
        bottom = df_plot[col]
    else:
        bottom += df_plot[col]

plt.title("Descomposición porcentual de latencia E2E percibida")
plt.xlabel("Goal ID")
plt.ylabel("Porcentaje (%)")

plt.legend()
plt.tight_layout()
plt.show()
```



9. Analisis del jitter

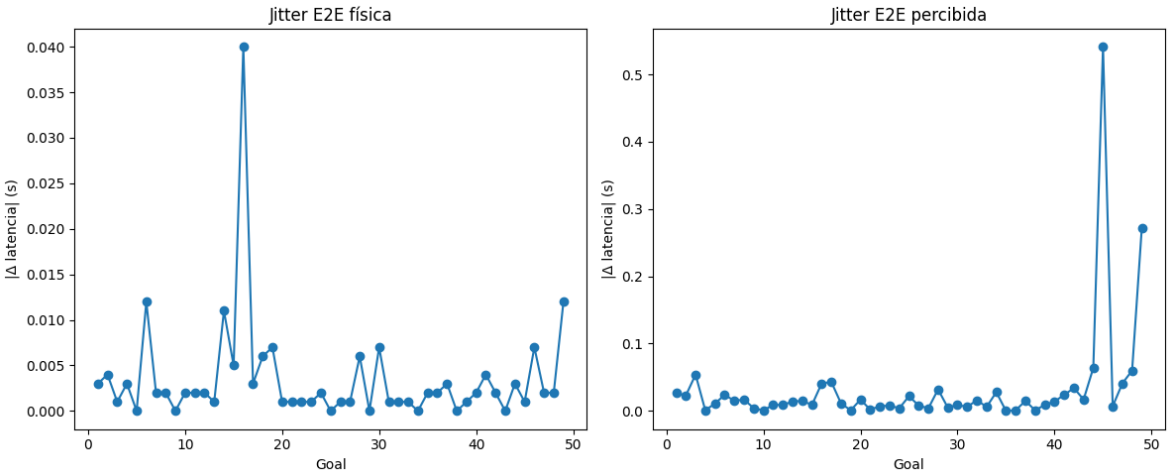
9.1 Metricas de jitter

In [30]:
```
cols = ["L_e2e_odom", "L_e2e_amcl"]

jitter_stats = {}

for col in cols:
    y = df_events[col].sort_index()

    jitter = y.diff().abs().dropna()

    jitter_stats[col] = {
        "mean": y.mean(),
        "std": y.std(),
        "cv": y.std() / y.mean(),
        "jitter_mean": jitter.mean(),
        "jitter_p95": jitter.quantile(0.95),
        "jitter_p99": jitter.quantile(0.99),
    }

jitter_df = pd.DataFrame(jitter_stats).T
print(jitter_df)
```

```
                mean       std        cv  jitter_mean  jitter_p95  jitter_p
99
L_e2e_odom  0.24948  0.053093  0.212813     0.055653      0.1324     0.136
08
L_e2e_amcl  0.71790  0.345731  0.481587     0.316041      1.1092     1.334
52
```

9.2. Jitter temporal

In [31]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

cols = ["L_e2e_odom", "L_e2e_amcl"]
titles = ["Jitter E2E física", "Jitter E2E percibida"]

for ax, col, title in zip(axes, cols, titles):

    y = df_events[col].sort_values().reset_index(drop=True)
    jitter = y.diff().abs()

    ax.plot(jitter, marker='o')
    ax.set_title(title)
    ax.set_xlabel("Goal")
    ax.set_ylabel("|Δ latencia| (s)")

plt.tight_layout()
plt.show()
```



Referencias

[1] Open Robotics, "Ros concepts," https://wiki.ros.org/ROS/Concepts, 2026, rOS Wiki, Accessed: 2026-03-05.

[2] iTEAM-UPV, "Private 5g network at upv," Universitat Politècnica de València, Tech. Rep., 2025, technical presentation, Autumn 2025.

[3] 3rd Generation Partnership Project (3GPP), "System architecture for the 5g system (5gs); stage 2," 3GPP, Technical Specification TS 23.501, 2025, release 20, Version 20.0.0.

[4] R. Lozano *et al.*, "Digital twin-based 6g testbed for immersive outdoor robotics: Capacity and latency analysis of the interaction with real and virtual objects," in *2025 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit)*. Poznan, Poland: IEEE, Jun. 2025, pp. 163–168.

[5] 3rd Generation Partnership Project (3GPP), "Study on edge computing in the 5g system architecture," 3GPP, Technical Report TR 23.758, 2021.

[6] A. Baxi, V. Mageshkumar, G. S. Murthy, A. Merwaday, A. Raju, G. Hosamane, S. Prabhu, V. Natarajan, S. Sudhakaran, D. Cavalcanti, A. N. Hatalkar, C. Maciocco, R. Iyer, and R. Vannithamby, "Achieving time-critical control in edge robotics systems over 5g," in *2025 17th International Conference on COMmunication Systems and NETworks (COMSNETS)*, 2025, pp. 739–747.

[7] K. Nishimura, T. Ishikawa, H. Sasaki, and S. Kato, "Raplet: Demystifying publish/subscribe latency for ros applications," in *Proceedings of the IEEE 27th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2021, pp. 41–50.

[8] M. Crespo-Aguado, R. Lozano, F. Hernandez-Gobertti, N. Molner, and D. Gomez-Barquero, "Flexible hyper-distributed iot–edge–cloud platform for real-time digital twin applications on 6g-intended testbeds for logistics and industry," *Future Internet*, vol. 16, no. 11, p. 431, 2024.